

UltraVec: Trellis-Coded Quantization for Calibration-Free Low-Bit Vector Search

Vladimir Čeperić

Abstract UltraVec is a calibration-free vector quantizer for low-bit approximate nearest-neighbor search that leads the data-oblivious field without a training corpus, a calibration pass, or any fitted state. It adapts trellis-coded quantization to database vectors: a free-start Viterbi encoder over a computed Gaussian codebook, scored by RaBitQ’s rescaled inner-product estimator. Its quality lever is trellis memory, which the stored record does not charge for: memories 9 through 16 serialize to the same bytes, and raising it from 2 to 16 state bits lifts direction fidelity from 0.876 to 0.970 and more than halves estimator deviation. Against seven data-oblivious comparators on SIFT, GIST and DBpedia–OpenAI it has the highest mean flat MIPS Recall@10 in eleven of twelve corpus–rate cells with paired intervals excluding zero against every comparator, and the highest Recall@100 and lowest reconstruction error in all twelve; on a shared footing with the pinned upstream E-RaBitQ library it leads by 4 to 7 points at every rate. The twelfth cell, two-bit GIST, ties at the reported draw and leads across five. On SIFT’s full 1,000,000-vector base all 25 pairwise intervals exclude zero, and on a million-vector IVF index it reaches Recall@10 .5545 against E-RaBitQ’s .4285. It matches or beats fitted 256-centroid PQ through 128 SIFT calibration vectors, needs no refit under covariate shift that costs transferred PQ 25.9–41.5%, and with a fitted PCA head exceeds the pinned E-RaBitQ library by 7.8 to 27.9 points for ten bytes.

Vladimir Čeperić
Department of Electronics, Microelectronics, Computer and Intelligent Systems,
University of Zagreb Faculty of Electrical Engineering and Computing,
Unska 3, 10000 Zagreb, Croatia
E-mail: vladimir.ceperic@fer.hr
ORCID: <https://orcid.org/0000-0002-7059-2369>

Keywords approximate nearest-neighbor search · vector quantization · trellis-coded quantization · compressed indexes · vector databases

1 Introduction

This paper presents UltraVec, a calibration-free quantizer for low-bit approximate nearest-neighbor (ANN) search that leads the data-oblivious field on three public corpora while fitting nothing to the data: no training corpus, no calibration pass, no per-corpus state. ANN systems compress database vectors because the uncompressed collection is often their dominant memory cost, and at one or two bits per dimension the quantizer also determines which true neighbors survive index navigation and candidate generation. Product Quantization (PQ) and its successors learn corpus-specific codebooks and are strong when representative training data are available. Data-oblivious schemes instead use fixed transforms and computed codebooks; they avoid calibration, simplify updates and multi-tenant deployment, and need no refit when the corpus distribution changes, and until now that simplicity has cost recall against well-fitted alternatives.

That cost is not structural: trellis-coded quantization (TCQ) [26] removes it. TCQ uses state carried between adjacent emissions, permitting a Viterbi encoder to search a much larger structured reconstruction set than a memoryless scalar quantizer at the same emitted rate. Trellises have recently proved useful for neural-weight compression [36, 35, 43], but that objective is weight distortion, not ANN ranking. UltraVec adapts the machinery to retrieval with a fixed Gaussian codebook, a free initial state, and the rescaled estimator introduced by RaBitQ [14]. The plain codec stores no fitted corpus state.

The central empirical observation is that trellis memory improves reconstruction-direction fidelity at a fixed number of emitted bits: sweeping the state memory from 2 to 16 bits on SIFT raises fidelity monotonically from 0.876 to 0.970 while the emitted rate never changes (Section 6). Direction fidelity is relevant because it controls the variance of the rescaled inner-product estimator under RaBitQ’s randomized-codebook model. For unit-normalized reconstructions it is exactly equivalent to MSE, so we do not claim an independent cross-codec effect. Instead, a byte-identical scale intervention isolates the role of scoring. The direction-aware principle itself is established in anisotropic vector quantization [16], and the estimator bound is due to RaBitQ [14].

Our contributions are:

1. **A retrieval-native, calibration-free trellis codec.** UltraVec combines a computed Gaussian reconstruction codebook, free-start Viterbi encoding, and rescaled inner-product scoring in a retrieval-native application of TCQ to ANN database vectors. The plain codec learns no corpus state. Trellis memory is a quality dial the stored record does not charge for: every memory from 9 to 16 serializes to the same bytes, and only encode time grows.
2. **A leading calibration-free operating point at low rates.** On hash-verified SIFT, GIST, and DBpedia–OpenAI inputs, UltraVec has the highest mean flat MIPS Recall@10 at the reported rotation in eleven of twelve corpus-rate cells, and the highest mean Recall@100 and lowest reconstruction error in all twelve. Paired query bootstraps give individually positive intervals against every available comparator in those same eleven cells; in the twelfth no comparator separates above UltraVec either, and none does in any cell on any corpus. On SIFT’s full 1,000,000-vector base the ordering holds at every rate, with all twenty-five pairwise intervals excluding zero. A five-draw rotation study (Section 5.4) scopes the closest cells on a second axis.
3. **Deployment evidence from cold start through fitted operation.** Six calibration draws place plain UltraVec level with or above every rate-matched PQ factorization through 32 SIFT calibration vectors and above every GloVe factorization at every budget, at a larger record; controlled covariate shift then costs PQ 25.9–41.5% and UltraVec nothing. A fitted PCA-head/residual extension beats the pinned upstream E-RaBitQ library at every tested rate while reporting its ten-byte record overhead;

warm PQ/OPQ comparisons identify the regimes where corpus fitting remains valuable.

4. **Mechanism, systems, and reproducibility evidence.** A fixed-emission-rate memory sweep establishes trellis state as a direction-fidelity lever; a byte-identical estimator-scale control lowers Recall@10 by 3.3–4.1 points for all three tested codecs while holding reconstruction fixed. On a million-vector SIFT IVF index UltraVec leads E-RaBitQ by 12.6 Recall@10 points at eight probes, and in a controlled HNSW study by 7.9 codes-only; scoring that graph from the codes alone cuts its resident memory to a third at unchanged recall. The artifact binds every reported result to clean-run, hash-validated evidence.

The paper targets a strong matched-emission-rate, no-calibration operating point and a retrieval-oriented use of TCQ; rate-distortion theory and the warm trained frontier remain complementary topics. The reference codes-only scorer streams its window directly from the packed codes, and from four probes on it outruns the fp32 reconstruction cache it replaces (Section 8); architecture-specific kernel optimization remains future work.

2 Background and Problem Setting

2.1 ANN with compressed vectors

Let $\mathcal{X} = \{x_1, \dots, x_N\} \subset \mathbb{R}^d$ be a database and q a query. We use d for the source dimension and $D \geq d$ for the transformed dimension after any zero padding. We study both cosine similarity and maximum inner product search (MIPS). A quantizer maps x to a compact code from which the index obtains an estimate $\hat{s}(q, x)$ of the selected score. For query q , let $G_k(q)$ denote the exact top- k set and $\hat{G}_k(q)$ the estimated top- k . We report

$$\text{R@}k = |G_k(q) \cap \hat{G}_k(q)|/k,$$

averaged over held-out queries. Flat recall ranks the complete database using only the codec’s directional reconstruction or native estimator. Index recall also includes the candidate set produced by IVF or HNSW. We do not apply exact reranking unless stated.

2.2 Fixed and fitted quantizers

A *data-oblivious* quantizer determines its transform and codebook without the target corpus. RaBitQ [14], TurboQuant [44], EDEN [39], and plain UltraVec use

this regime. A *data-dependent* quantizer estimates parameters from calibration data: PQ [20], OPQ [15], SAQ [23], and learned or additive variants occupy this regime. Data dependence introduces an explicit calibration and adaptation assumption. Our cold-start experiments vary the available calibration sample, and our shift experiment transfers a fitted PQ codebook without retraining. UltraVec’s optional PCA-head hybrid is also data-dependent; calibration-free claims apply to the plain codec. Throughout, *matched emission rate* means the same path bits per transformed coordinate. It does not imply identical complete records; start state, scaling, norm, and padding bytes are reported separately.

2.3 Rotate, quantize, estimate

Write $\hat{x} = x/\|x\|$ for the direction of a database vector x . The compared data-oblivious codecs first apply a seeded sign flip and Walsh–Hadamard rotation R to that direction, giving $u = R\hat{x}$. Rotation makes coordinates more nearly exchangeable and limits large-coordinate effects. The rotated vector is quantized to $\tilde{u}$. Only the direction is coded; the magnitude is carried separately. In the flat MIPS comparison, each codec uses its documented native directional score under the same preprocessing, and that score is multiplied by the exact stored norm $\|x\|$; the query norm is constant for a query and does not affect ranking. Estimator choice is therefore part of the measured codec. Section 6 controls that axis explicitly, and the official E-RaBitQ study is separated because it uses a distinct upstream estimator and search implementation.

For its native score, UltraVec uses the rescaled estimator introduced by RaBitQ. Write $o = u$ and $\bar{o} = \tilde{u}/\|\tilde{u}\|$. Their direction fidelity is

$$g = \langle \bar{o}, o \rangle.$$

Under RaBitQ’s randomized-codebook model, where the estimator is unbiased, its error standard deviation is [14]

$$\sigma_{\text{est}} = \frac{1}{\sqrt{D-1}} \sqrt{\frac{1-g^2}{g^2}}.$$

We use this existing result to motivate the trellis-memory sweep and then isolate estimator scaling with the byte-identical intervention of Section 6.

3 Trellis-Coded Quantization for Retrieval

UltraVec quantizes the rotated direction $u = R\hat{x}$ of Section 2 with a finite-state trellis. Its emitted bits are

the stored vector code; trellis memory changes the set searched by the encoder but not the emitted rate.

3.1 Computed Gaussian reconstruction values

At one trellis step, a window w containing the previous state bits and the new emission indexes a scalar reconstruction value. We compute rather than train these values. For unsigned 32-bit arithmetic, the fixed `lowbias32` permutation is

$$\begin{aligned} z_1 &= (w \text{ xor } (w \gg 16)) \cdot_{32} \text{7feb352d}, \\ z_2 &= (z_1 \text{ xor } (z_1 \gg 15)) \cdot_{32} \text{846ca68b}, \\ z_3 &= z_2 \text{ xor } (z_2 \gg 16), \quad h(w) = \frac{z_3 + 1/2}{2^{32}}, \end{aligned}$$

where $\cdot_{32}$ denotes multiplication modulo 2^{32} . Applying the inverse normal CDF and normalizing all 2^{M+b_s} scalar-table entries by their finite-table population mean and standard deviation gives

$$c(w) = \text{normalize}(\Phi^{-1}(h(w))).$$

The permutation assigns distinct uniform values to distinct windows; at the evaluated table sizes the resulting scalar entries are also distinct. The table is identical for every corpus and can be regenerated without data. The Gaussian target matches the approximately normal coordinates produced by the randomized Hadamard rotation.

This construction differs from the byte-sum “1MAD” (one multiply-add) reconstruction used in the neural-weight TCQ ancestor [36]. At the window sizes used here, 1MAD has many collisions. The inverse-CDF construction gives each reachable window a distinct value; Table 6 isolates the resulting recall improvement.

3.2 Free-start Viterbi encoding

Let the state contain the last M emitted bits. From state s , an emission e of b_s bits reaches

$$s' = ((s \ll b_s) \mid e) \bmod 2^M$$

and reconstructs the next coordinate from the window (s, e) . Dynamic programming selects the path minimizing total squared error. Concretely, let $y_i = u_i/\alpha$ where α is the standard deviation of that vector’s own rotated coordinates, computed per vector and stored in no corpus state. With $C_0(s) = 0$ for every state, the scalar recurrence is

$$C_{i+1}(s') = \min_{s, e: T(s, e) = s'} [C_i(s) + (y_i - c(s, e))^2].$$

The implementation enumerates states and emissions in ascending order, keeps the first equal-cost predecessor, uses 32-bit costs, and applies a total floating-point order when selecting the final state. Unlike a conventional forced-zero initialization, every state is a legal start. The selected start state costs $\lceil M/8 \rceil$ bytes and avoids a boundary penalty on the first coordinates.

Exact encoding evaluates 2^M states and 2^{b_s} outgoing emissions per step, for $O(D2^M2^{b_s})$ work per vector. A beam encoder retains the best W states and costs $O(DW2^{b_s})$. Encoding is an index-build operation; it is not performed for each query. At two bits and $M = 12$, width 16 reaches 0.5459 Recall@10 against exact encoding’s 0.6202; Appendix B shows that beam width, rather than larger memory, becomes the binding constraint.

3.3 Stored code and query score

For a scalar trellis ($b_s = b$), a vector stores Db emission bits, the start state, and a 32-bit rescaling factor. The per-vector footprint is therefore

$$B_{\text{code}} = \lceil Db/8 \rceil + \lceil M/8 \rceil + 4 \text{ bytes.}$$

The corresponding record rate is $8B_{\text{code}}/D$ bits per transformed coordinate. Increasing M across a byte boundary therefore changes total record rate even though the b -bit emission payload is fixed. MIPS experiments additionally store the exact database norm when it is not already part of a codec’s record. At query time, decoding gives the table-value vector $\tilde{u}$, and the stored scalar is $\rho = \langle \tilde{u}, u \rangle$. For $z = R(q/\|q\|)$, UltraVec estimates

$$\widehat{\text{cos}}(q, x) = \frac{\langle z, \tilde{u} \rangle}{\rho}, \quad \widehat{\text{sMIPS}}(q, x) = \|x\| \widehat{\text{cos}}(q, x),$$

where the omitted query norm is constant within a ranking. Thus the packed path is decoded in $O(D)$ time and scored with RaBitQ’s rescaled inner-product estimator. That estimator is unbiased under RaBitQ’s randomized-codebook model, in which the reconstruction’s orthogonal residual is conditionally isotropic. A Viterbi-selected reconstruction minimizes squared error over a structured path set, so that guarantee does not transfer directly. Random rotation alone does not establish conditional isotropy after path selection. We therefore measure the residual bias: its magnitude is at most 3.33×10^{-3} over the swept memories and 1.00×10^{-3} — about five percent of σ_{est} — at the $M = 16$ operating point (Section 6). The reconstruction table is shared and computed, with no learned per-vector or corpus codebook.

Table 1 Asymptotic costs of the plain scalar trellis.

operation	cost
exact encode	$O(D2^M2^{b_s})$ per vector
beam encode	$O(DW2^{b_s})$ per vector
codes-only score	$O(D)$ decode and dot product
serialized code	$\lceil Db/8 \rceil + \lceil M/8 \rceil + 4$ B
reconstruction cache	additional $4D$ B/vector
learned codebook	none

Two deployment modes must be distinguished. *Codes-only* scoring decodes the state path for each visited vector and preserves compressed storage. A *reconstruction-cache* mode decodes once at index build and retains D fp32 values per vector. It avoids repeated path decoding but adds $4D$ bytes per vector. The systems experiments report these modes and total resident memory separately. Table 1 collects the resulting encode, score, and storage costs.

3.4 Optional fitted head

The main method above is calibration-free. We also evaluate an explicitly data-dependent hybrid to test UltraVec as a residual coder. It fits the leading r principal directions of a corpus sample, stores quantized coefficients for that head, and trellis-codes the orthogonal residual. Rank and residual rate are selected on held-out database vectors used as pseudo-queries; the reported evaluation queries are disjoint. The warm comparison considers ranks $\{0, 8, 16, 32, 64, 96, 128\}$ and residual rates one through four under the stated byte budget, with eight-bit head coefficients. The upstream-library comparison instead fixes rank eight. This PCA-head residual hybrid can capture dominant variance directions and improve recall while storing fitted state that may require refitting after drift. Cold-start and no-refit claims apply to the plain codec.

4 Experimental Method

4.1 Public inputs and metrics

Table 2 summarizes the public inputs. Source objects are fetched from immutable or checksum-pinned URLs and are not redistributed. The artifact records source and derived-file SHA-256 values, byte lengths, licenses or license notes, and deterministic conversion commands. SIFT and GIST [21] use the published ANN-Benchmarks train/test arrays [7]. GloVe [28] uses the published angular split, whose train array holds 1,183,514 vectors; the cold-start experiment

takes the first 59,500 of them as its base and 200 queries. DBpedia–OpenAI [10] uses the first 100,000 rows of a pinned Hugging Face snapshot as base and the next 1,000 as queries, all of which are evaluated in the flat comparison. Non-power-of-two dimensions are zero-padded internally for the Hadamard transform — GIST from 960 to 1,024, DBpedia–OpenAI from 1,536 to 2,048, and GloVe from 25 to 32. Reported rates refer to the transformed dimension used by the code, which Table 2 lists alongside the source dimension.

The two SIFT counts separate the 1,000-query flat, shift and ablation experiments from the controlled 200-query tables: indexed search, cold start, and the pinned Faiss and ScaNN references, which use the same 200 queries as the codec IVF and HNSW harnesses they sit beside. The official E-RaBitQ comparison uses 1,000 queries.

The flat multi-rate experiments recompute exact raw-inner-product top- k over the loaded base and report the arithmetic mean of per-query recall at ranks 1, 10, and 100. Each codec estimates database-vector direction and its score is multiplied by the exact database norm, yielding a MIPS ranking; the query norm is ranking-invariant. Cold-start and indexed-search experiments use exact cosine gold. The distribution-shift study clusters raw SIFT descriptors, then normalizes each partition and uses exact cosine gold with full-precision normalized queries against each reconstructed database, as specified in Section 7. The pinned official E-RaBitQ comparison likewise uses normalized SIFT and the upstream inner-product mode against exact cosine gold. No result mixes metrics within an experiment. Unless a multi-seed experiment states otherwise, the seed is 42.

4.2 Comparisons

UltraVec runs at $M = 16$ in the comparisons the accuracy claims rest on — the flat tables, the million-vector scale point, the rotation-draw study and the IVF and HNSW studies — and at $M = 12$ in the supporting studies; every table states its own memory. The distinction costs no byte column: the start state occupies $\lceil M/8 \rceil$ bytes, two for every M from 9 to 16, so that whole range serializes to the same record. Every codec is charged its codes, one four-byte estimator scalar, and the exact database norm on the same terms; UltraVec’s only additional field is that two-byte start state, which the comparators cannot spend, their next rate rung costing a whole bit per coordinate. What the state costs is encode time, which grows as 2^M (Appendix B). Within the headline comparisons we report one memory rather than the best one per table, which

costs three cells — two-bit GIST by 0.7 points, two-bit DBpedia–OpenAI by 0.1, and the graph study — all reported rather than tuned around. Every other codec is likewise at one configuration throughout, chosen as the best its own byte budget admits, with one disclosed exception: BlockQuant’s block is bounded by a twelve-bit codebook-index cap that keeps its table trainable, which binds at four bits and halves the block there, so its four-bit row is a configuration limit of this harness rather than of the published method.

The flat MIPS table gives TurboQuant [44], RaBitQ/E-RaBitQ [14,13], UltraVec, the pyramid vector quantizer (PVQ) [11], EDEN [39], BlockQuant [6], and E8 [9] where supported the same database, queries, rate, deterministic rotation seed, and exact-norm treatment, while retaining the directional estimator each codec publishes: TurboQuant’s asymmetric score, EDEN’s own scaling, BlockQuant’s documented variant, RaBitQ’s rescaled one. PVQ and E8 are lattice and pyramid constructions that publish none, so both are scored with the same rescaled estimator as UltraVec, the neutral choice available. In this shared Rust harness, the combined column uses canonical one-bit RaBitQ and our implementation of the published E-RaBitQ construction at two through four bits. The separate upstream-library study pins RaBitQ-Library commit 540242ea0a68 (full hash in the manifest), applies a recorded one-line patch that fixes the upstream random rotation to seed 42, and uses the library’s E-RaBitQ outputs. The patch changes no quantization, scoring, or search logic, and two fresh preparation runs must produce identical output hashes.

The E8 row is an artifact-native, data-oblivious lattice baseline rather than a separately published ANN codec: each rotated eight-dimensional block is assigned to the nearest of the 2^{8b} lowest-norm E8 lattice points, normalized to unit per-coordinate variance, and scored with the same rescaled estimator. The codebook is the 2^{8b} lowest-norm points, so four bits would need 2^{32} , a 137 GB table; the tables stop E8 at two bits. The artifact implements three bits (2^{24} points, by closed-form nearest-point search), where E8 lands fourth on GIST and third on DBpedia and displaces no reported margin; those rows await a campaign re-run. E8 is the strongest comparator on GIST at low rate — UltraVec leads it at one bit and ties it at two — so its absence at four bits leaves that cell compared against a weaker field, not a defeated one. PVQ selects the supported block enumeration rate closest to each nominal target from above; this rate granularity is disclosed rather than treated as an exact-rate match.

Cold-start scores every codec by exact cosine over its decoded reconstruction. For PQ this is normalized re-

Table 2 Public evaluation inputs. “Base” is the largest base used; the flat multi-rate tables use 100,000 vectors, and the SIFT scale point of Section 5 uses all 1,000,000.

corpus	source d	transf. D	base	queries	role
SIFT	128	128	1,000,000	1,000 / 200	flat, shift, ablations / indexed, cold start
GIST	960	1,024	100,000	1,000	recall
GloVe	25	32	59,500	200	cold start
DBpedia–OpenAI	1,536	2,048	100,000	1,000	recall, warm baseline

constructed cosine rather than raw inner-product ADC; the choice keeps PQ aligned with the common cosine task and avoids a metric-induced reduction in its score. Warm comparisons use pinned **faiss-cpu** 1.14.2 and **scann** 1.4.2 wheels; their SHA-256 values and upstream license terms are in the baseline manifest.

4.3 Systems accounting

All systems measurements run on one host, an AMD Ryzen 9 7950X, with every run limited to 20 threads (recorded per bundle for both the OpenMP and Rayon pools). As in established recall-throughput evaluations [32], absolute QPS is machine-specific; we use it only within a table generated in one campaign. Every throughput point — codec harness, pinned Faiss and pinned ScaNN alike — is a single timed window that repeats the query batch until at least half a second has elapsed; the codec IVF and HNSW harnesses additionally require three full passes, so their window is the longer of the two conditions. The host is a shared-tenancy container, so points carry roughly ten percent run-to-run variance — visible in the retained Faiss sweep, whose QPS is non-monotone in n_{probe} over its three cheapest settings. We therefore report QPS to three significant figures, treat differences below about ten percent as indistinguishable, and use throughput together with recall and memory; the exact measured values are in the evidence bundles. Each window is a single campaign measurement, while the retained sweep illustrates run-to-run variation. The pinned Faiss and ScaNN rows are therefore presented as reference points. Serialized code bytes and total resident bytes are distinct. IVF reports both codes-only UltraVec and an fp32 reconstruction-cache point. HNSW reports the same duality: the reconstruction rows navigate from a resident fp32 array and isolate codec fidelity under one controlled navigation design, while the codes-only row builds the identical graph and then drops that array, scoring from the codes at query time. Its resident column counts the graph plus whichever of the two the configuration keeps.

4.4 Provenance and statistical protocol

The full campaign is accepted only when launched from a clean committed tree. Every evidence-producing command writes its stdout, stderr, dataset hashes, baseline pins, artifact commit and computation hash, toolchain, CPU, thread settings, and completion status to a machine-readable bundle. The claim validator accepts evidence from clean runs whose computation inputs, datasets, revision ancestry, and baseline pins match the recorded manifest. Appendix C records the bundle and pinned-library details. Confidence intervals in the cold-start study use six independent calibration draws and a Student t interval on that $n = 6$; they therefore describe calibration-draw variability at a fixed query set, not query-sampling error.

For the main flat comparison we use 10,000 paired query bootstraps over the per-query recalls, resampling queries and reporting the 95% percentile interval of the paired difference, and retain the resulting intervals; a cell is described as an UltraVec lead only when that interval excludes zero in its favor against every available comparator. Within one cell, this all-comparator claim is an intersection–union test across comparators and needs no comparatorwise multiplicity adjustment. Across cells, we report the number of individually positive cellwise decisions descriptively; we do not claim simultaneous or partial-conjunction control for the cell counts. Resampling queries also means the intervals cover query-sampling error only; the rotation is a second axis, swept separately where it could decide a cell (Section 5.4) and scoped in Section 10. The official E-RaBitQ comparison bootstraps 1,000 queries 2,000 times.

5 Data-Oblivious Codec Retrieval Accuracy

5.1 Flat MIPS comparison

Table 3 compares the available data-oblivious codecs at matched emission rates in a flat MIPS scan. Each codec’s native directional estimate is multiplied by the exact database-vector norm. UltraVec has the highest mean Recall@10 in eleven of the twelve corpus–rate

cells. At 2–4 bits its paired 95% interval against every available comparator individually excludes zero in its favor in eight of the nine cells; the margin over the closest comparator is 6.1, 5.3 and 4.5 percentage points on SIFT, 0.8 and 0.6 at three and four bits on GIST, and 1.1, 1.7 and 1.4 on DBpedia–OpenAI. These margins are the paired per-query differences at full precision, so they need not equal differences of the rounded cells in Table 3. The ordering is consistent; magnitude is corpus- and rate-dependent, and every sub-two-point margin here still separates. Each is a minimum over the whole field, so it shrinks as comparators are added.

Two-bit GIST is the one exception, and it is a four-way tie rather than a defeat: the paired intervals against E8, RaBitQ, EDEN and BlockQuant all span zero, E8’s at $[-0.87, +0.07]$, and in that same cell UltraVec leads E8 on Recall@100 (.956 against .949) and on reconstruction error (.067 against .094). The reported draw is also the worst of five for UltraVec in this cell: across the rotation study’s draws (Section 5.4) its mean is .9464 against E8’s .9408, and it leads E8 on the best draw (.955 against .948) and the worst (.937 against .927) alike, so the tie is a property of the draw rather than of the codec. Aggregated as the mean count is, eleven of twelve cells have individually positive all-comparator intervals — the same eleven — and no comparator separates above UltraVec in any cell on any corpus.

At one bit UltraVec leads on all three corpora, and on all three the lead survives the paired bootstrap: 4.32 points over TurboQuant on SIFT, 1.29 over E8 on DBpedia–OpenAI, and 0.77 over E8 on GIST with an interval of $[+0.17, +1.38]$. That last cell is the memory lever at its clearest. At $M = 12$ it was E8’s outright, at $M = 14$ a mean without a separation, and at the memory reported here a separation, which is the clearest single effect of spending state memory that the record does not charge for.

Metric choice matters. UltraVec has the highest mean in eleven of twelve cells at Recall@10, all twelve at Recall@100, and seven of twelve at Recall@1. TurboQuant’s MSE arm is ahead on one-bit SIFT and DBpedia–OpenAI, and the memory that buys the Recall@10 cells costs something at depth one; on GIST, where the whole field is near saturation and every codec exceeds .96 at Recall@1, E8 leads at two bits by .001 and PVQ at three by .003, while at four UltraVec and PVQ tie at .999. Appendix A tabulates both additional recall depths. We therefore scope the paired-lead claim to Recall@10 in this tested data-oblivious-codec field.

On SIFT at two bits, serialized MIPS sizes are 40, 40, 42, 41, 40, 40, and 40 bytes/vector for TurboQuant, E-RaBitQ, UltraVec, PVQ, EDEN, BlockQuant, and E8. TurboQuant already stores the exact database

norm in its record; the other directional codecs add a four-byte norm sidecar. UltraVec’s 42 bytes are its 38-byte cosine record — payload, free-start state and rescale factor — plus that sidecar, which is why Section 7 quotes 38 for the same codec; Table 16 reaches 42 because a centered record stores the residual norm instead. UltraVec also has the lowest reconstruction MSE in every one of the twelve cells, averaged over the whole loaded base, so the recall result is nowhere purchased by discarding reconstruction quality. Section 6 asks which statistic remains associated with recall across codecs and corpora.

5.2 The comparison at one million

The tabulated cells use 100,000-vector bases so that the three corpora stay matched. SIFT’s full base is ten times that, and repeating the comparison on all 1,000,000 vectors at the same rotation, rates and state memory leaves the ordering intact and widens it: UltraVec has the highest Recall@10 and Recall@100 at all four rates and the highest Recall@1 at three of four, with Recall@10 margins over the strongest comparator in each cell of +1.4, +7.5, +6.2 and +5.6 points at one through four bits, against +4.3, +6.1, +5.2 and +4.5 on the 100,000-vector base. Both lists are differences of the tabulated cells, so the three-bit figure here is the +5.2 that Table 3 shows rather than the +5.3 paired difference quoted above. At the 2–4-bit rates this paper reports the advantage therefore grows with scale rather than eroding, and at one million every rate separates: all twenty-five pairwise intervals exclude zero, one bit included, where the 100,000-vector base gives eight of nine.

5.3 Is the reimplemented comparator competitive?

The flat table’s closest comparator is an artifact-native implementation of the published E-RaBitQ construction, which invites the obvious question of whether that implementation is as good as the library. Section 7’s upstream experiment cannot answer it, because it changes metric, scorer and search path at once. Table 4 changes nothing but the implementation: the same normalized SIFT-128 base and 1,000 queries, the same exact-cosine gold, the same emitted rate, the same Recall@10. The upstream side is the pinned library’s own single-list output, which is an exhaustive scan under its native estimator rather than an approximate index.

Read naively, the artifact column looks like a weak baseline: it trails upstream by 4.3 to 9.0 points, and

Table 3 Flat MIPS Recall@10 on 100,000-vector public corpora (1,000 queries, raw inner-product gold, seed 42, UltraVec at $M = 16$). Bold is the highest mean in each corpus-rate cell; paired-bootstrap decisions are reported in the text. Transformed dimensions are in Table 2; all methods pay the same padding overhead. TQ is TurboQuant’s MSE-optimal reconstruction arm and TQ-IP its sketched inner-product construction, which needs at least two bits. “Ra/E-RaBitQ” denotes canonical RaBitQ at one bit and the artifact-native E-RaBitQ implementation at two through four bits. E8 is tabulated at one and two bits; its three-bit rows await a campaign re-run and four bits is beyond enumeration (Section 4). Unsupported cells are left empty rather than filled with an unmatched configuration. PVQ uses the nearest supported enumeration rate from above, as described in Section 4.

corpus	bits	TQ	TQ-IP	Ra/E-RaBitQ	UltraVec	PVQ	EDEN	BlockQuant	E8
SIFT	1	.280	—	.232	.323	.258	.232	.257	.253
	2	.287	.335	.519	.630	.556	.521	.544	.569
	3	.401	.324	.700	.802	.750	.712	.745	—
	4	.571	.438	.824	.896	.851	.835	.850	—
GIST	1	.870	—	.874	.892	.872	.874	.874	.884
	2	.900	.875	.937	.937	.931	.939	.935	.941
	3	.944	.905	.958	.974	.963	.965	.966	—
	4	.954	.946	.976	.985	.977	.977	.979	—
DBpedia–OpenAI	1	.738	—	.735	.753	.738	.735	.740	.740
	2	.755	.744	.853	.881	.851	.855	.862	.870
	3	.832	.761	.913	.942	.922	.916	.925	—
	4	.901	.837	.951	.968	.950	.950	.954	—

Table 4 Implementation parity on a shared footing (normalized SIFT-128, 1,000 queries, exact-cosine gold, UltraVec at $M = 12$). Only the implementation differs across columns. Centering is applied to both codecs or to neither. Bold is the highest value in each row.

bits	upstream	E-RaBitQ		UltraVec	
		artifact	+center	artifact	+center
1	.3598	.2805	.3674	.3100	.4181
2	.6285	.5389	.6301	.6202	.7057
3	.7870	.7158	.7790	.7996	.8412
4	.8706	.8279	.8669	.8912	.9124

every paired interval excludes zero. The cause is a centering convention, not implementation quality. RaBitQ quantizes a residual to a centroid, and a single-list upstream build centers on the global mean; the reconstruction path does not center at all, because a flat scan has no partition to center on. Supplying that one mean closes the gap at every rate: +0.76, +0.16, −0.80 and −0.37 points against upstream, with three of the four intervals spanning zero. The reimplement is faithful.

Applying the same mean to UltraVec — centering is a property of the comparison, so it is applied to both codecs or to neither — puts it 4.2 to 7.7 points above the pinned library at every rate, all intervals excluding zero. The lead therefore does not rest on the reimplement. Centering moves the margin in both directions: it widens it at one bit and narrows it by about two points at two through four, so the like-for-like number a deployment should expect is both codecs centered, 4.5 to 7.6 points over the library, rather than the flat table’s. We keep the uncentered convention in Table 3 because it is the correct one for a partition-free scan and is applied identically to every codec.

5.4 How much of a cell does the rotation draw decide?

Every codec here is a randomized construction sharing one seeded sign flip, so each reported cell is a single draw from a rotation distribution and the paired query bootstrap covers query sampling only. That is adequate for a wide margin and not for a cell decided by about a point. We re-run the flat comparison under five rotation draws, holding the query seed fixed so the two sources of variation do not mix. The draw moves every codec together, so the quantity of interest is UltraVec’s margin over each comparator, recomputed per draw. We report the closest one, and apply Section 4’s intersection–union rule to decide a cell: it counts only when every comparator’s draw interval excludes zero. A margin against the per-draw maximum over the field is also available, but it charges UltraVec for the size of the field rather than for anything about the codec, since the best of seven noisy values sits above any single comparator’s mean.

The sweep is scoped by draw dispersion rather than by margin. GIST is covered at every reported rate because it is the high-dispersion corpus: its two-bit cell is the one the query bootstrap already flagged, and its three- and four-bit margins are smaller than the draw-to-draw interval measured one rate below, so the bootstrap alone cannot carry them. DBpedia–OpenAI is covered at one and two bits; its remaining margins are +1.71 and +1.36, and every codec on that corpus moves by only 0.3–0.8 points across draws at the swept rates against 0.7–3.0 on GIST. We report that scope rather than extrapolate from it.

The sweep now runs at the memory the cells are reported at, so no operating-point gap remains. On DBpedia–OpenAI both cells separate against every

Table 5 UltraVec’s rotation-draw margin over its closest comparator across five draws (100k base, 1,000 queries, query seed 42, every row at $M = 16$, the operating point Table 3 reports). Coverage follows draw dispersion: GIST at every rate, DBpedia–OpenAI at the two rates that include its narrowest cell, and SIFT not at all, its margins being several points wide. The last column applies Section 4’s intersection–union rule to this axis: it is “yes” only when *every* comparator’s draw interval excludes zero.

corpus, bits	closest	margin (pp)	95% CI	all separate
GIST, 1	E8	+1.28	± 1.14	no
GIST, 2	E8	+0.56	± 1.60	no
GIST, 3	BlockQuant	+0.66	± 0.36	yes
GIST, 4	EDEN	+0.44	± 0.32	yes
DBpedia, 1	E8	+1.68	± 0.75	yes
DBpedia, 2	E8	+1.22	± 0.39	yes

comparator in every draw, the closest being E8 at +1.68 and +1.22 points. GIST separates the same way at three and four bits: every comparator’s interval excludes zero, and the codec that comes closest differs by rate, BlockQuant at three and EDEN at four. Four of the six swept cells therefore pass the same intersection–union test on the rotation axis that Table 3’s cells pass on the query axis.

The two low-rate GIST cells do not, and the reason is dispersion rather than sign. At four bits UltraVec’s worst draw against any comparator is an exact tie and never a deficit; at one bit three of six comparators overtake it in the worst of five draws, and at two bits two of seven, which is what the last column records. The two-bit cell is the one the query bootstrap already flagged. Across draws UltraVec has the highest mean of any codec there, .9464 against E8’s .9408, a margin of +0.56; but that margin’s draw interval spans zero and so does the paired query interval at the reported draw, so it is a mean lead on both axes and a separation on neither, and we claim it as exactly that. The one-bit cell is claimed on the query axis, where its interval against E8 excludes zero ($[+0.17, +1.38]$), and scoped on this one. What separates the low-rate cells from the rest is UltraVec’s own draw spread, 2.3 points at one bit against 0.4 at three and 0.3 at four: at low rate the cell is decided inside the transform’s own noise, and state memory is what narrows it.

We therefore claim the three- and four-bit GIST cells on both axes, the one-bit cell on the query axis alone, and not the two-bit one. A sub-point margin can still separate: whether it survives the transform is a separate question from how large it is, and it is worth measuring rather than inferring.

Table 6 Core component ablations at $M = 12$.

variant	2 bits	3 bits	4 bits
inverse-CDF code	.6202	.7997	.8912
byte-sum 1MAD	.6056	.7873	.8762
difference	+.0146	+.0124	+.0150
free start	.6202	.7997	.8912
fixed start	.5875	.7729	.8765
difference	+.0327	+.0268	+.0147
tail-biting	.5384	.7258	.8256
difference	−.0818	−.0739	−.0656

Table 7 Tail-biting against free start at two bits and $M = 16$, five rotation draws, 100k base, 1,000 queries. D is the transformed dimension; every row is UltraVec against itself, and the byte columns are the serialized record.

corpus	D	free B	tail B	mean Δ	min	max
SIFT	128	42	40	−4.78	−6.20	−4.10
GIST	1024	266	264	−0.06	−0.70	+0.80
DBpedia	2048	522	520	+0.00	−0.20	+0.20

5.5 Core component ablation

Table 6 changes one design choice at a time on SIFT-100k with $M = 12$, 1,000 queries, exact fp32-cosine gold, and seed 42. The inverse-CDF code improves Recall@10 by 1.2–1.5 points over byte-sum 1MAD across the tested rates. Free initialization helps most at two bits, where a forced boundary consumes a larger fraction of the path’s effective freedom. These experiments use a cosine scorer rather than the raw-inner-product scorer of Table 3, so only within-table differences are compared.

The free start is what the record’s $\lceil M/8 \rceil$ -byte start field pays for. Tail-biting removes it by constraining the path to end in the state it starts from: the state is the last M emitted bits, so a wrapped path’s start is a suffix of its own codes and is recomputed rather than stored, which puts the record at byte parity with RaBitQ. The wrap pins the last $\lceil M/b \rceil$ emissions, a fraction $M/(Db)$ of the payload, so its cost falls with dimension. Table 7 measures it at two bits and $M = 16$ on all three corpora across the rotation study’s five draws. On SIFT the wrap pins 6.2% of the payload and costs 4.8 points; on GIST and DBpedia it pins under one percent and the mean change is within a tenth of a point of zero. The lever is free on the embedding corpora and unfavourable on the low-dimensional one, and it is reported as an arm rather than adopted, so every other table keeps the free-start record.

6 Direction Fidelity and Recall

6.1 Hypothesis

Recall depends on whether score error reverses nearby pairs, not directly on average coordinate distortion. Under RaBitQ’s randomized-codebook model, Section 2 gives a standard deviation that decreases monotonically with direction fidelity g . This motivates

$$\begin{aligned} g \uparrow &\implies \sigma_{\text{est}} \downarrow \\ &\implies \text{fewer ranking inversions.} \end{aligned}$$

The first implication follows from the existing RaBitQ model [14]; the second is a ranking-model intuition whose empirical scope depends on the corpus and code family. Moreover, g and MSE are intrinsically related for normalized vectors, so their distinct associations require more than a marginal correlation.

UltraVec supplies a constructive reason to test the hypothesis: increasing trellis memory M enlarges the structured path set without changing emitted bits. In the ablation, Recall@10 increases from 0.5281 at $M = 4$ to 0.6202 at $M = 12$ on the same two-bit SIFT setting. This fixed-emission-rate quality knob carries an encoding cost that grows exponentially under exact Viterbi.

Table 8 measures the first link of the hypothesis directly, sweeping M at a fixed two bits per transformed dimension. Direction fidelity rises monotonically from 0.876 to 0.970 and the estimator’s empirical standard deviation falls by more than half, while the emitted rate is unchanged — only the start-state field grows, from one byte to two. E-RaBitQ, which has no memory lever, is re-measured at every M as a control and remains at $g = 0.9351$ (spread 0.0000), which shows the trend belongs to the codec and not to the harness. Trellis fidelity passes E-RaBitQ’s between $M = 2$ and $M = 4$. In this sampled SIFT setting we therefore observe $M \uparrow \Rightarrow g \uparrow \Rightarrow \sigma_{\text{est}} \downarrow$; the rest of this section examines the next step, from σ_{est} to ranking.

6.2 Why a pooled separation is not identifiable

For any unit-normalized reconstruction, MSE equals $2(1 - g)$ identically. Consequently, a pooled partial correlation cannot distinguish fidelity from MSE for the six normalized codec families in our comparison. The only departure is the frozen TurboQuant control, whose asymmetric scorer uses an unnormalized reconstruction; removing it makes the two diagnostics exactly anti-collinear and both partial associations zero. We therefore do not use a pooled cross-codec separation as mechanism evidence. The estimator-scale control below

Table 8 Direction fidelity versus trellis memory at a fixed emitted rate (SIFT-100k, two bits, 100 sampled queries against a seeded 1,000-vector database subsample, seed 42), swept through the $M = 16$ operating point the flat comparison reports. The emitted bit rate is identical in every row. Every entry here is a unit-normalized reconstruction, so the MSE column equals $2(1 - g)$ by construction and is shown for continuity with the other tables rather than as an independent statistic. Empirical σ_{est} is the pooled standard deviation of estimated minus exact cosine over the 100,000 query–database pairs; empirical bias is their pooled mean. The subsampling is also why this MSE differs in the fourth decimal from the full-corpus value quoted in Section 5.

M	start state (B)	g	σ_{est} (emp.)	bias (emp.)	recon MSE
2	1	.8761	.04357	+.00333	.24773
4	1	.9365	.03017	−.00008	.12707
6	1	.9553	.02413	+.00060	.08948
8	1	.9621	.02177	+.00088	.07581
10	2	.9658	.02104	+.00086	.06832
12	2	.9678	.02014	+.00083	.06448
14	2	.9691	.02014	+.00076	.06177
16	2	.9703	.01956	+.00100	.05936
E-RaBitQ control		.9351	.03071	−.00343	.12982

Table 9 Byte-identical estimator-scale control on three comparators (SIFT, two bits; the campaign ran UltraVec at $M = 12$, which the intervention does not touch).

codec	unbiased R10	biased R10	difference
EDEN	.525	.484	−.041
E-RaBitQ	.521	.488	−.033
BlockQuant	.549	.512	−.037

supplies the identifiable test by changing scoring while holding reconstruction, g , and MSE fixed.

6.3 Estimator-scale control

The primary ranking test is a controlled intervention on SIFT at two bits. For EDEN, E-RaBitQ, and BlockQuant, we keep the reconstruction byte-identical and change only the scoring multiplier from $1/\rho$ to the reconstruction-MSE-optimal value $\rho/\|\tilde{u}\|^2$, where $\rho = \langle \tilde{u}, u \rangle$. Table 9 shows the outcome. Recall decreases for all three: EDEN by 4.1 points, BlockQuant by 3.7 and E-RaBitQ by 3.3.

The three scalar codecs form a focused control set, so the result supports estimator choice as important while leaving broader code-family generalization open. The cross-codec diagnostic is consistent with this result only where reconstruction normalization breaks the exact fidelity–MSE identity; it is not independent evidence for the six normalized codecs.

6.4 Interpretation and ownership of ideas

This analysis builds on ScaNN’s anisotropic vector quantization, which weights error parallel to the datapoint direction in a trained loss [16]. The rescaled-estimator variance formula is RaBitQ’s [14]. Our specific contribution is to expose trellis memory as a fixed-emission-rate way of moving the direction-fidelity operating point and to isolate estimator scaling with a byte-identical intervention. The evidence is scoped to the tested codec set.

7 Calibration and Fitting Regimes

7.1 Cold-start calibration sweep

PQ must estimate a codebook before it can encode a database. To measure that cost, we draw K calibration vectors, train PQ, encode the full evaluation base, and rank by normalized reconstructed cosine. The evaluation base is 50,000 vectors and 200 queries on SIFT, and the 59,500-vector base with 200 queries on GloVe. We repeat each random draw with seeds 42–47. Plain UltraVec uses no draw and is therefore constant across seeds. Both codecs emit two bits per transformed dimension and both are padded from 25 to 32 on GloVe; UltraVec additionally stores its start state and rescaling scalar, which is the whole of the difference. The records differ in size: 38 against 32 bytes per vector on SIFT (+19%), and 14 against 8 on GloVe (+75%). The GloVe comparison therefore characterizes the calibration regime at these stated record sizes.

Table 10 and Figure 1 report selected points. On SIFT, UltraVec’s Recall@10 is 0.636. PQ is lower through $K = 128$, with confidence intervals below the fixed value, and crosses it at $K = 256$. PQ trained on the full 50,000-vector evaluation base reaches 0.715. On GloVe, UltraVec is 0.634; PQ improves from 0.288 at $K = 32$ to 0.568 with the full 59,500-vector evaluation base, while UltraVec remains at 0.634. That last point sits a shade below the 20,000-vector draw: the trainer holds its 25 Lloyd iterations fixed at every calibration size, so the largest fit need not be the best converged, and we report it rather than tuning the budget. The best point in the sweep is still 5.2 points short of the codec that fits nothing. Together with the SIFT crossover, these results locate the calibration regimes in which plain UltraVec and this PQ configuration lead.

The PQ trainer uses 256 centroids per subspace, 25 Lloyd iterations, seeded random-point initialization with replacement, and random-point reseeding of empty clusters. For $K < 256$, duplicated initial

Table 10 Cold-start Recall@10: PQ mean and 95% t interval over six calibration draws, UltraVec at $M = 12$. Δ is PQ minus plain UltraVec (SIFT .636; GloVe .634).

	SIFT			GloVe		
	K	PQ (95% CI)	Δ	PQ (95% CI)	Δ	
	32	.476 [.471,.482]	−.160	.288 [.277,.299]	−.346	
	64	.562 [.556,.568]	−.074	.374 [.367,.382]	−.260	
	128	.625 [.620,.630]	−.011	.454 [.444,.464]	−.180	
	256	.682 [.672,.692]	+ .046	.521 [.517,.524]	−.113	
	1,000	.703 [.694,.712]	+ .067	.552 [.545,.560]	−.082	
	20,000	.718 [.712,.723]	+ .082	.582 [.570,.593]	−.052	
	full	.715 [.715,.715]	+ .079	.568 [.568,.568]	−.066	

centroids are therefore unavoidable. The observed crossover at $K = 256$ is empirical rather than a sufficient-sample threshold; reliable codebook estimation generally needs more data. At $K = 0$, PQ has no fitted model. UltraVec’s advantage is therefore most relevant to a new or small per-tenant index; a growing index can later accumulate representative data and adopt the fitted operating point after training and re-encoding.

7.2 Does the crossover belong to PQ or to its factorization?

The sweep above fixes PQ at 256 centroids per subspace, which makes $K < 256$ an awkward regime by construction: below 256 draws the subquantizer cannot fill its codebook with distinct centroids. That configuration alone therefore cannot separate “PQ needs this many calibration vectors” from “a 256-centroid PQ does.” We hold the emitted rate fixed at two bits per dimension and vary the subquantizer width instead, so every arm spends identical bits per vector and differs only in how much it has to estimate.

Three widths exist in the byte-indexed class, and we run all of them. A c -bit subquantizer needs c to be a whole multiple of the two bits per dimension, or the emitted rate is no longer exactly two bits; and it needs its $c/2$ -dimensional subspaces to divide $D = 128$. Within $c \leq 8$, one index byte per subquantizer, these admit $c \in \{2, 4, 8\}$ — 4, 16 and 256 centroids over 128, 64 and 32 subspaces. The intermediate cardinalities are not omitted but unreachable: a 64-centroid subquantizer would need three-dimensional subspaces, and 128 does not divide by three. Wider subquantizers meet the rate condition — $c = 16$ gives 16 eight-dimensional subspaces at the same 256 bits — but need more centroids than any calibration draw here has vectors, so the class is bounded at one index byte.

Table 11 shows the crossover is a property of the factorization, reading every cell against the same six-

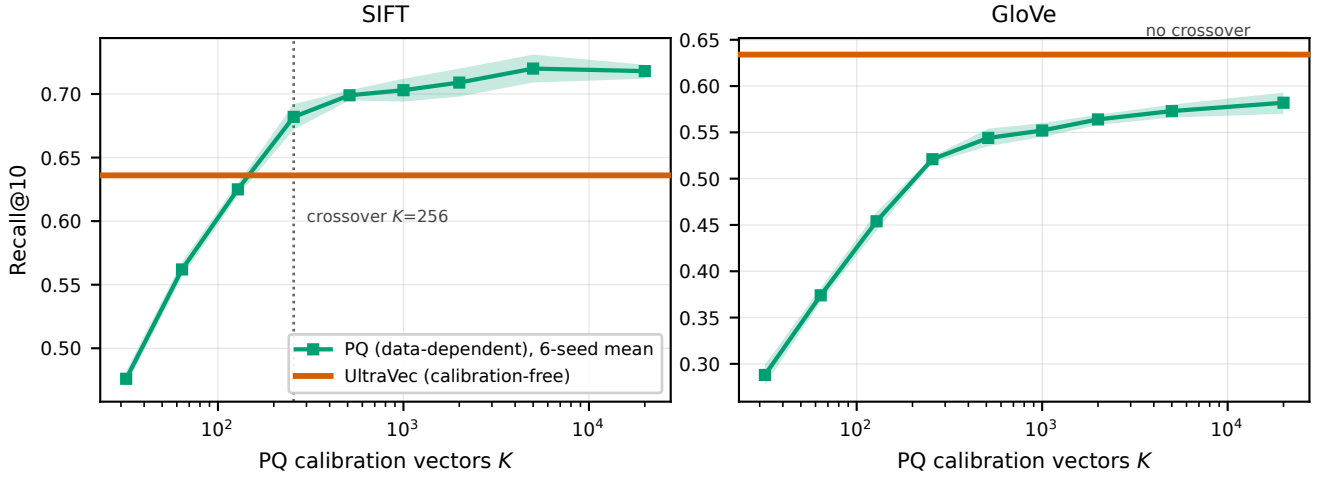


Fig. 1 Cold-start sweep: PQ Recall@10 as a function of calibration size K (six-calibration-draw mean; the shaded band is the 95% t interval) against the constant calibration-free UltraVec value. Both codecs emit two bits per transformed dimension. On SIFT, PQ crosses UltraVec at $K = 256$ — the first tested calibration size not smaller than its 256 centroids per subspace. This is an empirical crossover, not a claim that one vector per centroid is sufficient training. On GloVe, PQ improves throughout and remains below plain UltraVec through the full 59,500-vector evaluation-base point for the tested PQ configuration. Tabulated in Table 10.

Table 11 Cold-start crossover against PQ factorization at a fixed two-bit emitted rate (SIFT-50k, 200 queries, six calibration draws, UltraVec at $M = 12$). Calibration-free UltraVec is .6360 throughout. Bold marks the first K at which PQ’s mean leads; which of those are separations is given in the text.

K	256 ctr./32 sub.	16 ctr./64 sub.	4 ctr./128 sub.
32	.4765	.6255	.6348
64	.5620	.6480	.6448
128	.6252	.6595	.6543
256	.6820	.6722	.6625
1,000	.7028	.6852	.6735
full	.7150	.6930	.6740

draw interval the section uses at $K = 32$. Throughout this section a cell where one method’s mean is higher is described as a lead, and one where the interval also excludes the other value as a separation; the two differ in exactly the cells below. The conventional 256-centroid arm trails plain UltraVec through $K = 128$ — separated at every one of those sizes — and passes it at 256, reproducing the sweep above. The 16-centroid arm separates at $K = 64$; at $K = 32$ it is UltraVec that leads it, and separately so. The 4-centroid arm is level at $K = 32$ (.6348 against .6360) and still level at $K = 64$, where its interval [.6335, .6561] contains the calibration-free value, so its first separated lead is $K = 128$. The honest statement is therefore that UltraVec leads the conventional configuration through 128 vectors and the middle one through 32, and is level with the most frugal factorization through 64.

The frugality is bought, not free. At full calibration the same arms reach .7150, .6930 and .6740: each halv-

ing of the codebook that lowers the crossover also lowers the plateau, by 2.2 and 4.1 points against the conventional arm. No single PQ configuration dominates the range, so a deployment that cannot predict its calibration budget must choose which end to lose at. Plain UltraVec needs no such choice, which is the property this section is about — and on SIFT it sits below every arm’s warm plateau, which Section 7.6 prices.

The same control on GloVe reaches the opposite conclusion, and running it was not optional: the sweep above is precisely the reason to distrust a GloVe arm measured at one factorization. Table 12 sweeps all three at $D = 32$, where the widths become 8, 16 and 32 subspaces. No arm crosses plain UltraVec’s .6340 at any calibration size, including full, and under the same interval test every one of those cells is a separation rather than a tie. The closest any PQ configuration comes is .5817 at $K = 20,000$, a size Table 12 omits, still 5.2 points behind, and the low-cardinality arms that rescued PQ on SIFT are further behind here rather than closer: at full calibration they reach .5540 and .5220 against the conventional arm’s .5680. The ordering by cardinality is the same as on SIFT — smaller codebooks start higher and plateau lower — but on this corpus the whole family plateaus below the calibration-free codec.

Two things separate the corpora. GloVe-25 is the harder retrieval task, and at $D = 32$ a two-bit PQ record is 8 bytes against UltraVec’s 14, a 75% premium that Section 7 states and that this table does not remove. The result is therefore that UltraVec leads every rate-matched byte-indexed factorization through-

Table 12 Cold-start crossover against PQ factorization on GloVe at a fixed two-bit emitted rate (GloVe-59.5k, 200 queries, six calibration draws, UltraVec at $M = 12$). Calibration-free UltraVec is .6340 throughout. Unlike Table 11 there is no bold entry: no arm leads at any K .

K	256 ctr./8 sub.	16 ctr./16 sub.	4 ctr./32 sub.
32	.2882	.4737	.4873
64	.3745	.4905	.5030
128	.4538	.5203	.5092
256	.5207	.5253	.5163
1,000	.5523	.5408	.5258
full	.5680	.5540	.5220

out the GloVe sweep at a substantially larger record, not that PQ is beaten at matched bytes.

7.3 A controlled SIFT covariate shift

Cold start varies calibration quantity while drawing from the deployment corpus. We separately test transfer. Mini-batch k -means with two clusters partitions SIFT1M; the smaller cluster A is the training region and the larger cluster B supplies a 513,178-vector deployed database plus 1,000 held-out queries. Their mean per-coordinate separation is 1.304 standard deviations and their centroid cosine is 0.4086. The partitions are then normalized, and exact cosine top-10 is recomputed within B .

PQ trained and deployed on B reaches 0.6335 Recall@10 with a 16-byte code and 0.7682 with a 32-byte code. Transferring the same configurations from A changes recall to 0.3705 and 0.5690: absolute reductions of 0.2630 and 0.1992, or 41.5% and 25.9% relative. Plain UltraVec requires no fit and obtains 0.5820, exceeding both transferred points – the 16-byte one by 0.2115 and the 32-byte one by 0.0130. Each method compresses only the database; queries remain full-precision normalized vectors.

UltraVec’s two-bit emission payload is 32 bytes, while its actual serialized record is 38 bytes after the free-start state and rescale factor; both the corrected shift script and Table 16 use the full accounting. This experiment therefore supports a measured transfer-loss/no-refit result at the stated 32- and 38-byte records. Fitted methods can detect drift and retrain; the experiment measures the pre-retraining regime. UltraVec’s fitted PCA-head residual hybrid lies outside this scope.

Table 13 Cost of the centering mean (SIFT-128, two bits, exact-cosine gold, six draws, UltraVec at $M = 12$). $K = 0$ is the plain calibration-free codec and needs no draw; the pinned upstream library scores .6285 on this footing. The mean is a single D -vector, 512 bytes on SIFT, counted as shared model state, plus the four per-vector bytes the residual norm needs.

K	R@10	95% CI half	vs upstream (pp)
0	.6202	—	−0.83
8	.6892	.0123	+6.07
16	.6985	.0080	+7.00
32	.7016	.0025	+7.31
64	.7031	.0031	+7.46
256	.7057	.0029	+7.72
full	.7057	—	+7.72

7.4 The cheapest fit: what a mean costs and buys

Section 5.3 showed that centering the reconstruction on the corpus mean lifts UltraVec well above the pinned upstream library. A mean is corpus state, so a centered codec is *not* calibration-free and we do not report it as though it were. It belongs on the same calibration axis as PQ, and the question that decides whether it is worth having is how little data it needs. Estimating the mean from a K -vector draw puts it on exactly the axis Section 7 uses for PQ, with the same six draws and the same interval convention.

Eight vectors buy 6.07 of the 7.72 available points, and 32 buy 7.31 with the draw interval already down to a quarter point; beyond 256 the estimate is indistinguishable from the whole-corpus mean. Set against Section 7.2, where PQ needs 256 calibration vectors to pass plain UltraVec and 64 even in its most frugal rate-matched factorization, the contrast is not that one method calibrates and the other does not. It is that the calibration cliff sits where the quantity being estimated puts it: roughly an order of magnitude lower for one mean vector than for a codebook, at the same emitted rate.

This gives three priced operating points rather than a binary. Plain UltraVec learns nothing and is within a point of the upstream library; a mean from eight vectors passes it by six; the fitted PCA-head hybrid of Section 7.5 passes it by 19.1 at the same rate and carries a ten-byte per-vector overhead. A deployment can pick its rung and know what it pays.

The middle rung is not free. Centering costs 512 bytes of shared model state on SIFT — one D -vector, amortized over the corpus and negligible per vector — but it also costs four bytes *per vector*, because the decomposition $u = c + \|u - c\|r$ codes only the unit residual and the norm has to be stored. On a 38-byte two-bit record that is over ten percent. The codec charges it wherever it reports its own record, which is why the

centered IVF rows of Table 16 are 42 bytes rather than 38; the warm table is the stated exception, where the centered column reuses the plain column’s record so that the two stay comparable. Every codec pays the scalar identically, so no comparison moves; what moves is the absolute record, which is the number a deployment budgets against.

Finer centering is worth more than the rung above it costs. Replacing the single mean with k -means centroids — each vector coded as the residual to its own cluster centre, the regime the pinned library ships in — adds only the assignment index on top of that norm, one byte at 256 centroids. Measured on the flat comparison’s SIFT base at two bits and $M = 12$, it lifts the whole field: the trellis goes from .6200 uncentered to .6640 at 256 centroids and .6780 at 1,024, while its margin over the best comparator stays in the same band, +5.10 uncentered against +5.90 at either granularity. That it lifts every codec is the point — like the rotation, it is shared preprocessing rather than a property of one construction. The single global mean is the configuration that does *not* survive this footing: it costs the field 8 to 10 points under raw inner-product gold, where the centroid is a poor summary of a corpus that has not been partitioned, even though it gains 8.55 under the exact-cosine gold of Table 13. Which centering helps is a question about the metric, not about the codec.

Two limits on the claim. The comparison centers UltraVec and E-RaBitQ but not the remaining comparators, so it establishes a margin over the pinned library and not over the field; Table 3 remains the field comparison, uncentered throughout. And the sweep is one corpus at one rate under cosine gold. The multi-draw evidence is two-bit SIFT only; the other rates are not swept and are not claimed.

7.5 Fitted UltraVec versus the pinned upstream library

The shared flat table does include E-RaBitQ: the artifact-native Rust baseline uses canonical RaBitQ at one bit and the published E-RaBitQ construction at two through four bits, putting the comparison codecs behind one common interface. Table 14 complements that comparison by using the pinned upstream RaBitQ-Library with its own E-RaBitQ estimator and search path. It compares the data-dependent PCA-head residual hybrid from Section 3.4 with the pinned official library in its matched single-global-head setting. It is a 100,000-vector normalized-SIFT experiment with 1,000 queries, exact-cosine top-10 gold, and the official library’s inner-product mode. The PCA-head side ranks normalized reconstructed vectors

Table 14 Official E-RaBitQ comparison on SIFT-100k (Recall@10; 2,000 query bootstraps; UltraVec at $M = 12$). Values are populated from the validated clean campaign. Rates are matched; the byte rows count emission payload plus the hybrid’s ten-byte head/state overhead, not a common serialized scorer layout.

method	1 bit	2 bits	3 bits	4 bits
E-RaBitQ	.3598	.6285	.7870	.8706
bytes/vector	16	32	48	64
UltraVec+PCA-head	.6384	.8193	.9052	.9483
bytes/vector	26	42	58	74
difference	+.2786	+.1908	+.1182	+.0777

by exact cosine; UltraVec+PCA-head is higher at all four rates. Query bootstrap confidence intervals for the Recall@10 difference exclude zero at every rate: [.268,.290], [.182,.200], [.111,.125], and [.072,.083] at one through four bits. The comparison matches the emitted rate, while the stored representations differ: the rank-8 fitted head and the free-start state together add ten bytes per vector (eight and two respectively), so the hybrid spends 26–74 bytes against E-RaBitQ’s 16–64. The relative byte premium is largest where the recall gap is largest, and both are reported together. The ordering survives it: read one rate down, where the hybrid is byte-matched or smaller, it still leads at .6384 against .6285, .8193 against .7870 and .9052 against .8706. Part of the gap is the centering the head also supplies: at the same 42 bytes centered plain UltraVec reaches .7057 (Table 13), so at two bits the head adds 11.4 points over the cheapest fit where it adds 19.1 over the pinned library. These figures count emission payload and, for the hybrid, its head and start state; they do not claim a common serialized layout for the distinct scorers. The pinned build centers on the base mean, which Section 7.4 counts as corpus state, so the like-for-like plain comparison is Table 4’s centered row, which leads the library at every rate. The official repository’s unseeded sign rotation is fixed only to make the comparison repeatable; the patch and its hash ship with the artifact.

7.6 Warm data-dependent boundary

Table 15 places the method beside fitted PQ/OPQ as warm accuracy reference points. Plain UltraVec emits four bits per transformed coordinate. The hybrid selects its residual rate and head rank within a $d/2$ -byte budget, where d is the source dimension; the fitted reference uses $d/2$ two-dimensional PQ/OPQ subquantizers at eight bits each. These are nearby-budget rather than byte-matched points. Every margin below is stated against that table’s fitted-reference column, which is

Table 15 Warm cosine Recall@10 on 1,000 queries with serialized bytes in parentheses, over 100,000-vector bases, UltraVec at $M = 12$. Plain UltraVec uses four emitted bits per transformed coordinate; the hybrid is selected within a $d/2$ -byte budget; PQ/OPQ uses $d/2$ eight-bit subquantizers. The fitted reference is the better of PQ/OPQ where both are run. Record sizes are shown because they are not uniform across methods. The centered column shares the plain column’s per-vector record and adds one D -vector of shared model state, which the fitted columns’ codebooks are likewise not charged. Bold is the highest recall in each row. The byte conventions differ by four bytes between the plain and PCA-head columns: the plain-UltraVec figure includes the 4-byte rescale factor its inner-product estimator needs, whereas the PCA-head figure counts the trellis payload, free-start state and quantized head only, which are what the cosine reconstruction compared here requires.

corpus	fitted ref.	plain	centered	PCA-head
SIFT	.9069 (64)	.8912 (70)	.9124 (70)	.9073 (58)
GIST	.8929 (480)	.9060 (518)	.9606 (518)	.9302 (450)
DBpedia	.9684 (768)	.9648 (1,030)	.9827 (1,030)	.9578 (642)

PQ on all three corpora. The lead varies by corpus. On GIST, plain UltraVec is 1.3 points above the reference and the fitted hybrid is 3.7 points above it. On SIFT, plain UltraVec is 1.6 points below the reference while the fitted hybrid is level with it. On DBpedia–OpenAI, the reference is 0.4 point above plain UltraVec and 1.1 points above the hybrid.

The plain column is the odd one out here, and deliberately so: it is the only entry in the table that fits nothing, while PQ and OPQ train codebooks and the hybrid fits a projection. This is the regime where fitting is permitted, so the centered column gives UltraVec the cheapest thing it can learn — one corpus mean, the lever Section 7.4 prices — and leaves the per-vector record untouched. It is enough to change the ordering: 0.5, 6.8 and 1.4 points above the fitted reference on SIFT, GIST and DBpedia–OpenAI, and above the fitted hybrid on all three. The two cells where plain UltraVec trailed a fitted codec were regime mismatches rather than codec deficits.

These results support the paper’s regime distinction: data-oblivious and fitted coding provide complementary operating points, and on this table the distance between them is one vector wide. The fitted references use 64, 480, and 768 bytes on SIFT, GIST, and DBpedia; the hybrid uses 58, 450, and 642 bytes. Plain UltraVec uses 70, 518, and 1,030 bytes after start-state, rescale, and transform-padding overhead. The DBpedia plain-codec point consequently has a substantially larger record, four bytes of which are the inner-product rescale this cosine comparison does not use.

8 Indexed Search and Systems

8.1 IVF recall, throughput, and memory

The IVF experiment indexes all one million SIFT base vectors with 1,024 coarse lists, and every codec that supports it quantizes the residual to its list centroid (Section 4). UltraVec appears three times in Table 16: scanning from a reconstruction cache, scanning from the codes alone — these two produce the same ranking by construction — and the codes with the one-bit companion shortlist of Section 9 prefiltering each list, which is an approximation and is reported with its own recall.

The resident column is computed the same way for every codec: serialized codes, plus shared model state, plus each codec’s own per-entry array. Charged naively the trellis would be the only total carrying its container, at more than a kilobyte per vector, but three of those components are not per-vector cost. The reconstruction table is a pure function of the rate and the memory, so one shared copy serves the index rather than 1,024 identical copies across 1,024 posting lists; the entry arrays are reserved exactly rather than grown by doubling; and the entry struct’s three side tables — reconstruction cache, residual sketch, sign code — are all empty here, so behind one nullable pointer they cost nothing rather than 72 bytes per vector recording their own absence beside a 42-byte record. The entry is then 64 bytes rather than 128, and the resident total 109. What remains is real — the entry is still wider than a scalar quantizer’s 48 — which is why its total continues to exceed theirs, by 21%, at a serialized record two bytes larger.

Throughput compares kernels as much as codecs, so every compared scan gathers and accumulates through the same runtime-detected AVX2/FMA kernels. The exception is TurboQuant, which keeps the scalar scan of the frozen upstream implementation it copies; that understates it, and it is the weakest row on recall by a wide margin, so the omission decides nothing. At $n_{\text{probe}} = 8$ the exhaustive codes-only scan leads E-RaBitQ by 12.6 points, 0.5545 to 0.4285, at 0.78 times its throughput, and by 14.8 on peak recall, 0.5990 to 0.4510. The shortlist is a curve rather than a point, so the table sweeps its size: $C = 128, 256$ and 512 give 0.5305, 0.5425 and 0.5465 at 5,870, 5,150 and 4,860 queries per second, so four times the reranking budget costs seventeen percent of the throughput and returns 1.6 points. Every point on that curve leads E-RaBitQ on both axes at once, by 10.2 points at 1.26 times its throughput at $C = 128$. Its first stage keeps the query in full precision, which is worth up to 6.8 points of shortlist quality at a scan three

to five times slower than the binarized alternative; Appendix B measures both ends of that trade, and which one to deploy depends on whether throughput or recall is the binding constraint.

The codes-only scan looks as though it must be serial and is not, and the distinction decides what the throughput column measures. Writing the state recurrence out, $s_{i+1} = w_i \bmod 2^M$ with $w_i = (s_i \ll b_s) \mid e_i$, the state is exactly the last M emitted bits, so w_i is the $(M + b_s)$ -bit field at offset ib_s of the stream $[s_0 \parallel e_0 \parallel e_1 \parallel \dots]$ — a window at a known offset rather than a value carried from the previous step. Every w_i is therefore a pure function of the packed bits and the table entries they index can be gathered in any order. This is not a property we introduce: it is the reason QTIP adopted the same bitshift transition, which it contrasts with a generic trellis where “recovering the state ... requires a graph walk” [36]. UltraVec inherits that transition (Section 9), so it inherits the property.

Emitting the window stream first and then gathering makes codes-only scoring one vectorized pass instead of a scalar unpack, a chain walk materializing $\hat{u}$, and a separate dot. What remains is a genuine cost of a different kind: the reconstruction table holds 2^{M+b_s} floats — 1 MiB at the two-bit operating point reported here — so the scan gathers from a table far larger than the sixteen-entry register lookup that shuffle-based scans over product codes rely on [4, 5]. That is why this scan sits below a scalar quantizer’s, and it is a property of coding with a 2^{M+b_s} codebook rather than of the trellis being sequential. The reconstruction cache stops paying once the scan dominates. It is ahead at the two shallowest probes, by 19,000 against 16,100 at one and 10,400 against 9,470 at two, where the walk it skips is a larger share of the work. From four probes on it reaches the same recall at lower throughput than scanning the codes (3,300 against 3,640 at $n_{\text{probe}} = 8$), and it touches 621 resident bytes per vector where the codes touch 109. We keep its row because caching reconstructions is the arm a reader expects to be faster, and showing that it is not is the clearest consequence of the scan being a stream rather than a walk.

8.2 Controlled HNSW navigation

Table 17 evaluates HNSW as a codec drop-in: a graph is rebuilt from each codec’s reconstructions with graph degree $M_{\text{HNSW}} = 16$ and $ef_{\text{construction}} = 200$. The implementation keeps those fp32 reconstructions resident for both construction and navigation. At $ef = 64$, UltraVec reaches 0.5240 Recall@10 against E-RaBitQ’s 0.4440 at comparable QPS (9,940 against 10,500), while total resident memory is 770 and 766 bytes/vector respectively.

Recall here is a property of the reconstructions alone, so it is unmoved by the scan work of Section 8.

That resident figure is an artifact of navigating from reconstructions, and the codes-only row removes it. The graph is built exactly as above — every vector reconstructed, every insertion made against those reconstructions — and the fp32 array is dropped before the first query, leaving search to score candidates from the codes. The graph is therefore the same object as the fp32-navigating UltraVec row, and only the scan differs. Resident memory falls from 770 bytes/vector to 258, below the fp32 control’s 734 rather than above it, while Recall@10 is 0.5225 against 0.5240 and throughput rises to 11,300 queries per second from 9,940. The build column is the one place the rows differ: the harness encodes the base again to populate the codes-only scorer once the graph exists, so it covers two encodes where the row above covers one.

Throughput rising while work per candidate goes up is the same reversal Section 8 reports for the IVF reconstruction cache: 512 bytes per vector of fp32 is a large object to walk randomly, and not walking it buys more than decoding costs. This is the configuration a compressed graph would actually deploy, and it is the only row here that is compressed in RAM rather than only on disk. The reconstruction rows are kept because they isolate codec fidelity under one controlled navigation design, which is a different question from what a deployment stores.

8.3 Pinned mature-system references

Table 18 gives pinned Faiss IVFPQ and ScaNN points on the same SIFT1M source and campaign machine. Faiss uses IVF1024,PQ32; ScaNN uses a trained 1,024-leaf tree — matching Faiss’s n_{list} — with 4-bit anisotropic hashing at two dimensions per block, and the swept parameter is the number of leaves searched, the analogue of n_{probe} . Both have an approximately 32-byte payload. ScaNN reaches 0.6245 Recall@10 at 10,700 QPS on one thread; Faiss reaches 0.4370 at 9,100 on twenty. The throughput columns are therefore not comparable to each other, and ScaNN’s is a floor: its recall column, which is what the warm concession rests on, is unaffected by threading and reproduces exactly. These are useful warm-system reference points; their training, index structures, candidate-generation policies, and optimized kernels differ from the controlled codec harness, so the table is interpreted as a set of reference operating points.

Two reproducibility observations guide that interpretation. The ScaNN driver seeds partitioning,

Table 16 SIFT1M IVF at two bits (20 threads, UltraVec at $M = 16$). QPS values are campaign-machine measurements, rounded to three significant figures (exact values in the evidence bundles). Bold marks the best recall and the best throughput in each result column; where a reconstruction-cache and a codes-only row tie, the codes-only row carries it, being the one a deployment stores. “Peak R10” is the maximum over $n_{\text{probe}} = 1, \dots, 128$. The shortlist arm reranks the C best sign-code matches in each probed list; at $n_{\text{list}} = 1,024$ the mean list holds 977 vectors, so $C = 128, 256$ and 512 rerank about 13%, 26% and 52% of a probed list. Its first stage scores those codes against the full-precision query and reapplies the stored residual norm; both choices are measured in Appendix B.

codec	regime	code B	resident B	R10/QPS at probe 1	R10/QPS at probe 8	peak R10
TurboQuant	codes	40	82	.1210 / 15,100	.2000 / 3,700	.2000
E-RaBitQ	codes	40	90	.2145 / 19,500	.4285 / 4,660	.4510
UltraVec	fp32 cache	42	621	.2660 / 19,000	.5545 / 3,300	.5990
UltraVec	codes	42	109	.2660 / 16,100	.5545 / 3,640	.5990
UltraVec	codes + shortlist ($C=128$)	58	125	.2445 / 21,900	.5305 / 5,870	.5790
UltraVec	codes + shortlist ($C=256$)	58	125	.2580 / 21,900	.5425 / 5,150	.5910
UltraVec	codes + shortlist ($C=512$)	58	125	.2615 / 20,400	.5465 / 4,860	.5930
PVQ	codes	41	115	.2380 / 15,100	.4765 / 2,780	.5115

Table 17 Controlled SIFT1M HNSW at $ef = 64$ (20 threads, 200 queries, UltraVec at $M = 16$). QPS rounded to three significant figures; throughput is similar at this operating point. The resident column counts each vector once: for fp32 the stored record *is* the navigation cache, so it is not charged twice. Bold marks the lowest resident memory and the highest throughput among the compressed rows. “Build s” is whole-pipeline index construction, encode included, and it predates the encoder work of Appendix B.3: it is an upper bound on what the shipped encoder costs, and Table 21 gives that encoder’s current rate.

codec	code B	resident B	Recall@10	QPS	build s
fp32	512	734	.9280	10,600	514
UltraVec	38	770	.5240	9,940	2,241
UltraVec, codes	38	258	.5225	11,300	3,754
E-RaBitQ	36	766	.4440	10,500	572
TurboQuant	40	737	.1785	9,370	544

hashing and sampling and verifies that two independent processes return identical neighbors at every setting; re-running the campaign reproduced its recall column exactly. The Faiss index carries no such guarantee: its k -means runs multi-threaded, and a re-run on the same host with the same seeds moved its whole recall curve by about 1.3 points (0.4500 to 0.4370 at $n_{\text{probe}} = 64$). Its cheapest operating points are also the least stable in throughput, spanning a $2.1\times$ range across $n_{\text{probe}} \in \{1, 2, 4\}$ and running non-monotonically in n_{probe} there. We therefore quote Faiss to the precision the measurement supports and present both rows as reference points.

Table 18 Pinned mature-system references on SIFT1M (200 queries). Faiss runs at 20 threads, as the codec harnesses do; the pinned ScaNN wheel’s batch entry point is single-threaded and its parallel path is not used, so its throughput column is a one-thread measurement and understates what that library reaches at 20. Payload excludes index/model overhead; QPS is campaign-machine specific and rounded to three significant figures (exact values in the evidence bundles). ScaNN’s build is verified deterministic across independent processes; Faiss varies by about one recall point between campaigns.

system	search setting	Recall@10	QPS
Faiss IVF1024,PQ32	$n_{\text{probe}} = 8$	.4145	28,600
Faiss IVF1024,PQ32	$n_{\text{probe}} = 64$	.4370	9,100
ScaNN tree+AH	searched leaves = 8	.5565	50,100
ScaNN tree+AH	searched leaves = 64	.6245	10,700

9 Related Work

9.1 Fixed quantization for ANN

RaBitQ provides a randomized binary quantizer, an unbiased distance estimator, and an error bound for ANN search [14]; its extension develops asymptotically optimal multi-bit constructions and improved estimators [13]. TurboQuant studies online quantization with fast randomized transforms [44]. EDEN develops optimal-scale scalar quantization in a distributed-estimation setting [39]. Block-Sphere Vector Quantization (“BlockQuant”) extends the fixed-codebook comparison to optimized small blocks [6], while classical pyramid vector quantization provides the spherical PVQ baseline [11]. IVF-TQ shares this paper’s deployment premise most directly: it builds an inverted file whose residual layer is a fixed rotation followed by a precomputed Lloyd–Max scalar quantizer, so there is no codebook to fit and none to retrain as the corpus grows [30]. Avoiding calibration is therefore not by itself a distinguishing property in 2026; avoiding it

at no recall cost is. The difference is what the codec does with its rate: IVF-TQ quantizes each coordinate independently, whereas UltraVec carries state between emissions, which is what makes trellis memory a quality lever at an unchanged emitted rate. A recent symmetric study compares RaBitQ and TurboQuant directly under matched theory and experiments and reports RaBitQ ahead in most tested settings [12]; we retain each as an independently configured baseline. UltraVec differs structurally by carrying state between emissions, giving it a stateful codebook distinct from memoryless scalar quantization and independent block lattices. The flat MIPS experiment holds inputs, rate, rotation seed, and exact-norm treatment fixed while retaining each codec’s native directional estimator; its shared harness uses artifact-native implementations of canonical RaBitQ at one bit and E-RaBitQ at multi-bit rates. The separate upstream-library experiment keeps E-RaBitQ’s upstream estimator and search path intact.

9.2 Trellises and lattices

TCQ is due to Marcellin and Fischer [26], who applied Ungerboeck’s set-partitioned trellis codes [37] to source rather than channel coding: a Viterbi encoder searches a state-expanded reconstruction set at an unchanged emitted rate. Trellises have also been used to quantize high-dimensional *directions* under an alignment objective: noncoherent TCQ encodes a channel direction on the Grassmannian by exploiting a duality with non-coherent sequence detection, selecting codewords by normalized inner product with an off-the-shelf Viterbi search [8]. That work establishes the principle that a trellis can supply a structured directional code at a fixed feedback rate, which is the same principle UltraVec relies on. It is limited-feedback beamforming rather than retrieval: there is no database, no query workload, and no ranking metric, and the estimator is not RaBitQ’s. We therefore claim the retrieval construction and its evaluation, not the idea that trellises can code directions. Trellis-coded residual vector quantization has appeared in image coding [22], as has projected residual quantization for ANN search [40]. Both place their coder *after* a residual decomposition and optimize squared error over that residual. UltraVec instead runs the trellis directly on the rotated unit vector with a universal projection-free construction and a rescaled inner-product estimator, targeting database-vector ranking. Our fitted PCA-head hybrid adopts a residual decomposition, and it is labelled data-dependent throughout. QTIP and QuIP# brought trellises, incoherence processing, and structured codebooks to

neural-weight compression [36,35]. Those works optimize model-weight distortion under offline calibration. The line has since continued: Proteus removes runtime table lookups with a computed generator and a per-layer Gaussianizer [43], and BCJR-QAT relaxes trellis-coded weight quantization into a differentiable training objective [18]. The move toward computed, lookup-free codebooks is therefore shared rather than ours alone, which sharpens the distinction instead of blurring it: those systems still fit per-layer statistics to a fixed weight matrix and minimize weight distortion, whereas the plain UltraVec codec fits no corpus state at all and is scored for retrieval ranking. More specifically, UltraVec’s bitshift state transition originates in Mao and Gray’s random-permutation trellis coder [25] and was reused by QTIP with Hadamard incoherence processing and computed Gaussian codes. UltraVec changes the finite reconstruction mapping, uses free-start encoding, and targets database-vector ranking with an ANN score estimator. E8 and nested-lattice methods provide complementary rate-distortion references [9,29,27].

9.3 Fitted vector quantization

PQ and OPQ learn subspace codebooks and rotations [20,15]. Recent systems increase expressiveness with implicit neural codebooks (QINCo and QINCo2) [17,38], adaptive segmentation and bit allocation (SAQ) [23], projection to information-dense dimensions (MRQ) [42], individualized non-uniform codes (NVQ) [34], and streaming-update objectives (CoDEQ) [1]. These form the relevant warm, data-dependent frontier. Our evaluation focuses on the calibration regime through a standard fitted codec, while the PCA-head hybrid belongs to this fitted family. MRQ fits a projection and quantizes an information-dense leading subspace while summarizing the tail; SAQ fits PCA-space segmentation and bit allocation. Plain UltraVec instead retains every transformed coordinate, carries state across emissions, and fits no corpus projection. These are complementary trained and calibration-free design points; the flat experiment isolates data-oblivious codecs under a shared interface.

9.4 Direction-aware scoring

Anisotropic vector quantization in ScaNN showed that MIPS quality depends on the orientation of quantization error and incorporated that fact into a learned loss [16]; anisotropic additive quantization extends the idea

[45]. UltraVec uses RaBitQ’s estimator analysis to motivate direction fidelity, demonstrates trellis memory as a construction knob, and isolates scoring with a byte-identical estimator-scale control.

9.5 Indexes and deployed systems

HNSW [24], DiskANN [33], IVF, and their production implementations separate candidate navigation from vector scoring. Locally adaptive quantization shows how a fitted codec can be integrated into update-friendly compressed indexes [2, 3]; GPU-native IVF-RaBitQ demonstrates that fixed quantization can also be engineered at high throughput [31], and training-free binary quantization has been used to reshape graph topology itself [41]. VIBE emphasizes controlled, reproducible evaluation across modern embedding workloads [19]. Our pure-Rust IVF/HNSW implementations serve as diagnostic controls, alongside pinned Faiss and ScaNN warm-system reference points.

Two ideas from the SIMD-scan line bear on how a trellis is served. Product Quantization Fast Scan and Quick ADC make the per-subspace lookup a register-resident shuffle over sixteen entries, transposing codes into blocks of database vectors and quantizing distances to eight bits [4, 5]. That instruction is unavailable to us: a trellis window indexes 2^{M+b_s} values, orders of magnitude beyond a shuffle’s capacity, and the published route for larger tables — splitting them into register-sized pieces recombined by blends — costs more per lookup as the table grows. Section 8 reports the scan under that constraint rather than claiming it away. Their two-stage shape does transfer: a cheap approximate pass shortlisting candidates for exact rescoring applies to any codec with a cheap sketch, and we evaluate it there using a one-bit sign code over the same rotated vector.

We are unaware of an earlier application of trellis-coded quantization to ANN/MIPS database-vector scoring, and that is what we claim. Neither trellis coding of directions [8] nor a calibration-free codec for vector search [44, 30] is new on its own, and the stateful computed codebook is QTIP’s [36]. What we claim is their combination for database-vector compression and ranking — the free-start Viterbi encoder over a computed Gaussian table scored by RaBitQ’s rescaled estimator — together with its retrieval evaluation and an empirical fixed-emission-rate direction-fidelity control — a quality lever no memoryless data-oblivious codec has, because it costs no stored byte.

10 Threats to Validity and Limitations

10.1 Scope of accuracy claims

The flat MIPS Recall@10 ordering is consistent on three public corpora, and Recall@100 and reconstruction error lead in all twelve cells. Two boundary cases: two-bit GIST, whose interval against E8 includes zero, and Recall@1, at seven of twelve rather than eleven. The Recall@1 leaders differ on one-bit SIFT and DBpedia–OpenAI, where the field is far from saturated and the state memory that buys the Recall@10 cells costs depth one, and on two- and three-bit GIST, where it is; four-bit GIST is a tie at .999. Appendix A tabulates both complementary depths. Native estimators can change the ordering, and the flat MIPS and official E-RaBitQ studies use different metrics and search paths, so they answer different questions. The fitted PCA-head results characterize the fitted extension; calibration-free claims apply to the plain codec.

10.2 Rates and padding

The Hadamard implementation pads non-power-of-two dimensions (Table 2). Every codec in the flat table pays the same padding, but the physical payload per source dimension is therefore higher than the nominal transformed-coordinate rate, especially for DBpedia. Storage tables report bytes so the overhead stays visible; GloVe appears only in the cold-start study, whose byte premium Section 7 states instead. Mixed-radix or blocked transforms are a direct production path for eliminating this padding. On DBpedia, padding adds 128 bytes at two bits and 256 bytes at the four-bit warm point. The resulting 1,030-byte warm record is larger than the 768-byte fitted PQ reference but remains roughly six times smaller than the 6,144-byte fp32 source vector.

10.3 Implementations of the compared codecs

The shared flat harness uses artifact-native implementations of every codec so that inputs, preprocessing, rate accounting, and interfaces remain common. This includes canonical RaBitQ at one bit and E-RaBitQ at multi-bit rates. These implementations follow the published descriptions, and Section 5.3 verifies the closest of them against the pinned library on a single shared task: given the same centering convention, the artifact-native E-RaBitQ matches upstream within a point at every rate. That check is what licenses the shared-harness rows; without it a reader could

not distinguish a faithful reimplementation from a weakened one. It covers E-RaBitQ only; the remaining comparators rest on their published descriptions and on the configuration choices Section 4 states, of which BlockQuant’s twelve-bit codebook-index cap is the one that binds, halving its block at four bits. Tuning matters: the final PVQ block selection raises SIFT Recall@10 by 20 and 27 points at three and four bits over the earlier setting, which is why each comparator is given the best its own budget admits.

10.4 Statistical interpretation

The paired intervals resample queries and therefore quantify query-sampling error. Every data-oblivious codec here is a randomized construction sharing one seeded sign flip and Hadamard rotation, and the reported cells are a single draw from that rotation distribution. Section 5.4 measures that second axis where draw dispersion is largest: GIST at every reported rate, and DBpedia–OpenAI at one and two bits, both at the $M = 16$ the cells are reported at. Four of the six swept cells separate against every comparator in every draw. Two-bit GIST does not and is not claimed; one-bit GIST is claimed on the query axis and scoped on this one, where three of six comparators overtake it in the worst of five draws. Two gaps remain. SIFT was not swept, its margins being several points wide, and neither were DBpedia–OpenAI at three and four bits, whose margins are +1.71 and +1.36 points. What the swept evidence supports is narrower than a guarantee: DBpedia–OpenAI’s narrowest cell is one of the two swept and led every draw, every codec on that corpus moves by at most 0.8 points across draws, and on GIST dispersion falls as the rate rises. A reversal at the unswept rates is therefore unlikely, but that is an inference from the swept ones rather than a measurement. Direction fidelity and MSE are algebraically equivalent for unit-normalized reconstructions. A pooled cross-codec partial correlation would therefore be identified only by the one unnormalized control, not by independent variation among the other six families, so we do not present it as mechanism evidence. The estimator-scale control supplies the direct intervention: all three tested codecs decrease by 3.3–4.1 Recall@10 points under the MSE-optimal biased scale. RaBitQ’s unbiasedness guarantee assumes its randomized-codebook model and does not transfer automatically to a Viterbi-selected trellis path. We claim no replacement theorem and no multi-corpus tail bound; the measured bias at $M = 16$ is $+1.0 \times 10^{-3}$, below the E-RaBitQ control’s -3.43×10^{-3} .

10.5 Calibration and shift

The SIFT crossover is a property of the PQ factorization rather than of PQ (Section 7.2), and the control sweeps the byte-indexed factorizations, those whose subquantizer index fits one byte; wider ones meet the rate condition but need a codebook larger than the calibration budget this section studies. On GloVe no factorization crosses at any calibration size, so that arm rests on no configuration choice either — but it does rest on a 75% larger record, which Section 7 states and this control does not remove. Learned quantizers, transferred codebooks and online adaptation can all reduce the calibration cost. The shift study uses one two-cluster partition of SIFT and measures an unretrained deployment, at 32 bytes for PQ against 38 for UltraVec.

10.6 Systems generality

Absolute QPS comes from one CPU limited to 20 threads and one reference implementation. The codec rows are artifact-native Rust implementations, not upstream production kernels, so their relative QPS is not an architecture-optimized ceiling. What that relative comparison now controls for is the optimization level of the scans themselves: every codec in Table 16 but one scores through shared runtime-detected AVX2/FMA kernels, so a throughput ordering cannot be an artifact of which codec’s scan happened to be hand-tuned. TurboQuant is the exception, still scalar as the frozen control (Section 8). The trellis row is understated in the other direction: it divides by its stored rescale per candidate where the scalar codecs multiply by a precomputed reciprocal, so its throughput column is also a floor. An upstream RaBitQ deployment would add the packed register-shuffle layout of the FastScan line [4,5], which this harness does not implement; E-RaBitQ’s peak recall over the whole probe sweep is 0.4510 against 0.5990, so recall rather than throughput decides that table. Our shortlist arm is not that technique: it is the same two-stage idea applied to a one-bit sketch, and it leaves the exhaustive scan it prefilters unaccelerated. What bounds the codes-only UltraVec scan is the gather from a 2^{M+b_s} -entry table, not a state dependency (Section 8), which is also why the reconstruction cache earns its place only at the shallowest probe. The HNSW study measures controlled navigation quality with resident reconstructions for every codec. Broader hardware, billion-scale [32], GPU, update, energy, and tail-latency studies remain future work. The reported HNSW build time covers the whole pipeline rather than encode-only ingestion. Appendix B gives beam-width/recall and memory/recall

sweeps alongside the encode-throughput grid over the same memories and rates (Table 21), so the quality a memory tier buys can be read against what it costs to ingest; the measured step from $M = 14$ to $M = 16$ at two bits is $4.20\times$, the $4\times$ the recursion implies, while the same step at four bits costs $9.62\times$ because the reconstruction table leaves cache. That grid is one corpus on one machine. Encode is the axis on which this codec asks the most: exact Viterbi is exponential in the memory that supplies its quality, so continuous rather than batch ingestion can drop to $M = 12$ for the same 22-byte record, seventeen times the encode rate, and 1.25 points of Recall@10 at one bit. The graph study is where that price is most visible: at $M = 16$ the HNSW index takes 2,241 seconds to build against 622 at $M = 12$, because construction is encode-dominated.

It is also where state memory stops paying. Recall@10 at $ef = 64$ falls by 0.75 points for the reconstruction arm and 0.30 codes-only, and the codes-only throughput advantage over the reconstruction arm narrows from 47% to 14%. The mechanism is the one Section 8 identifies as bounding that scan: it gathers from a 2^{M+b_s} -entry table, which grows from 64 KiB to 1 MiB here and leaves cache. Memory buys resolution in a linear scan and costs locality in a graph walk. We report one operating point rather than tuning M per table, so this is disclosed rather than selected around.

10.7 Data coverage

SIFT and GIST are established image-descriptor benchmarks; GloVe provides a low-dimensional case; and the DBpedia vectors come from one embedding model. Future coverage can extend to current multimodal and domain-specialized workloads, including VIBE’s modern embedding suites [19]. VIBE’s OOD setting draws queries and the corpus from different distributions; that query-shift question complements rather than duplicates our database-codebook transfer experiment, which changes the corpus between fitting and deployment. The 100,000-vector flat subsets and the GloVe evaluation base are deterministic prefixes: this makes them exactly reproducible but can preserve source ordering, so randomized or disjoint-subset sensitivity is useful future coverage for the corpora that are subset. The SIFT scale point uses the entire base and carries no prefix, and the ordering there is the same. Dataset license ambiguity also prevents redistribution of SIFT/GIST vectors, so reproduction depends on their continued availability from the public benchmark host.

11 Conclusion

UltraVec establishes trellis-coded quantization as a strong design point for low-bit vector search. A computed Gaussian reconstruction table, free-start Viterbi encoding, and rescaled inner-product scoring give the plain codec a large structured codebook without training on the target corpus. Trellis memory then improves that codebook at a fixed emitted rate: from 2 to 16 state bits, direction fidelity rises from 0.876 to 0.970 while empirical estimator deviation falls by more than half. This is a quality control no memoryless data-oblivious codec has, and it is free in the record: memories 9 through 16 serialize to the same bytes, so the lever costs encode time and nothing stored.

The public-data campaign shows that the resulting codec leads where low rate and limited calibration matter most. Across SIFT, GIST, and DBpedia–OpenAI, UltraVec records the highest mean flat MIPS Recall@10 at the reported rotation in eleven of twelve corpus–rate cells, with the highest mean Recall@100 — and the lowest reconstruction error — in all twelve. Paired query bootstraps give individually positive intervals against every available comparator in those same eleven cells; the twelfth, two-bit GIST, is a tie with E8 at the reported draw and a mean lead over it across five draws, and no comparator separates above UltraVec anywhere. A five-draw rotation study at the same operating point scopes that ordering: four of its six swept cells separate against every comparator in every draw, and the two that do not are GIST’s at one and two bits. On SIFT’s full 1,000,000-vector base all twenty-five pairwise intervals exclude zero. The six-calibration-draw cold-start study places it above the conventional 256-centroid PQ through 128 SIFT calibration vectors, level with or above every rate-matched factorization through 32, and above every rate-matched byte-indexed factorization throughout the GloVe sweep at a 75% larger record. Under controlled SIFT shift, transferred PQ recall declines by 25.9–41.5% relative to its in-distribution level, while plain UltraVec remains immediately deployable because it has no fitted state. For warm deployments, the fitted PCA-head extension beats the pinned upstream E-RaBitQ library at every tested rate while reporting its ten-byte record overhead, and the PQ/OPQ comparisons identify the complementary regimes in which corpus fitting remains valuable.

The mechanism and systems results turn these accuracy gains into a fuller ANN picture. Raising trellis memory directly improves direction fidelity, and switching three byte-identical codecs from unbiased to MSE-optimal estimator scaling reduces Recall@10 by 3.3–4.1

points. Fidelity and MSE are structurally equivalent for normalized reconstructions, so the controlled result—not a pooled separation—supports direction-preserving estimation as a useful design principle. On a million-vector IVF index UltraVec reaches .5545 at eight probes against E-RaBitQ’s .4285, and in a controlled HNSW study .5225 against .4440; scoring that graph from the codes alone holds that recall at 258 resident bytes rather than 770, a mode only the trellis arm implements. Optimized codes-only kernels and padding-free transforms are the clearest next engineering steps. The standalone, hash-locked artifact contains the implementation, experiment drivers, baseline and dataset pins, and claim-bound evidence needed to reproduce every reported result from a clean revision.

Statements and Declarations

Funding. No funding was received for this work.

Competing interests. The author declares no competing interests.

Author contributions. Vladimir Čeperić conceived the study, developed the method and software, designed and validated the experiments, analyzed the results, and wrote and reviewed the manuscript.

Data availability. The study uses public SIFT, GIST, GloVe, and DBpedia–OpenAI inputs. The artifact does not redistribute them. Its dataset manifest records the upstream URLs, snapshot identifiers where available, exact byte lengths, and SHA-256 values. It also records license information and the deterministic conversion steps for every evaluated file.

Code availability. The paper-specific artifact is publicly available at <https://github.com/vceperic/ultravec-ann>. It contains the UltraVec implementation, locked dependencies, container recipe, experiment drivers, baseline pins, result bundles, and claim validator. An archival identifier will be added after acceptance.

Ethics approval and consent. Not applicable. The study involves no human participants, animals, interventions, or private personal data.

A Recall at Every Reported k

Table 3 gives Recall@10; Table 19 gives the other two reported depths, from the same runs and bundles. The three produce different leaders: UltraVec has the highest mean in eleven of twelve cells at Recall@10, all twelve at Recall@100, and seven of twelve at Recall@1, where the differences are mainly on GIST—every codec exceeds 0.96 there at every rate, so the ordering turns on a handful of queries.

B Additional Trellis Design Ablations

All ablations use SIFT-100k, 1,000 queries, exact fp32-cosine gold, seed 42, and Recall@10. They differ in absolute level from the public-test experiment in Table 3, which scores against raw inner-product gold rather than exact fp32 cosine; comparisons within each ablation share the same queries and scorer. The core codebook and free-start ablation is reported in Section 5; this appendix retains the encoder and design-space details needed to justify the selected operating point.

B.1 Beam approximation

At two bits and $M = 12$, exact Viterbi reaches 0.6202. Beam widths 4, 8, and 16 reach 0.4326, 0.5080, and 0.5459.

We also test whether a beam can combine larger state memory with lower encoding cost. Beam encoding costs $O(DW2^{b_s})$ independent of 2^M , so memories beyond the exact-search range are computationally accessible. Sweeping $M = 14$ through 22 at $W = 64$ and $W = 128$, both sequences vary by about one point without trend in M and are separated by W alone—0.5874 to 0.5983 at the narrower width and 0.5971 to 0.6066 at the wider—while exact $M = 12$ reaches 0.6202, above every beam cell. The beam is the binding constraint at these widths: it offers an encode-time trade, and memory at fixed W adds no measured quality.

B.2 Memory sweep

Figure 2 shows state memory improving recall at a fixed emitted rate across the whole tested range. At two bits, the 32-byte emission payload is unchanged; the complete cosine record is 37 bytes for $M \leq 8$ and 38 bytes for $M \geq 10$ because the start state crosses a byte boundary. Returns flatten at $M = 12$: $M = 14$ adds 0.15 points over $M = 12$ at two bits for 4.11× the encode time (Table 21), so $M = 12$ is the supporting-study point.

That selection is made at two bits, and it does not transfer. The start state occupies $\lceil M/8 \rceil$ bytes, so memory comes in byte tiers and every M from 9 to 16 costs the same two bytes; within a tier, more memory is free in record size and paid for only in encode time. Table 20 sweeps that tier at one bit, where the codec is weakest and where fixed overhead is largest—6 bytes of a 22-byte record against 6 of 70 at four bits.

Returns have not flattened at one bit. Raising memory to $M = 14$ gains 1.14 points at an unchanged 22-byte record—nearly eight times the gain the same step buys at two bits—and $M = 16$ adds 0.11 further, still for free in the record. Under this cosine-gold sweep the tier looks saturated by 14; under the flat comparison’s own protocol (Section 5) the step to 16 is worth considerably more, which is why it is the reported operating point. The opposite move is a poor trade: $M = 8$ drops into the one-byte tier and saves a byte, but costs 2.75 points, which is worse per byte than simply paying for the state.

State memory is therefore free in the record and paid for in encode time. The start state occupies $\lceil M/8 \rceil$ bytes, two for every M from 9 to 16, so every memory in that range serializes identically and the comparison stays byte-matched; what grows is encode cost, as 2^M , tabulated below. We take that trade to its byte-free limit in the flat and systems comparisons and report $M = 16$ there (Section 4).

This table measures SIFT under exact-cosine gold, so it could not by itself move a reported cell. Section 5 settles the

Table 19 Flat MIPS Recall@1 and Recall@100 on the same runs as Table 3 (100,000-vector corpora, 1,000 queries, raw inner-product gold, seed 42, UltraVec at $M = 16$). Each cell is Recall@1 / Recall@100; leaders are given in the text, since the two depths disagree. Unsupported cells read “—”.

corpus	bits	TQ	TQ-IP	Ra/E-RaBitQ	UltraVec	PVQ	EDEN	BlockQuant	E8
SIFT	1	.243/.378	—/—	.165/.348	.237/.438	.198/.368	.165/.348	.204/.371	.198/.362
	2	.167/.413	.281/.434	.427/.618	.516/.716	.466/.646	.428/.620	.424/.641	.467/.657
	3	.303/.542	.201/.449	.593/.775	.741/.855	.678/.812	.630/.784	.669/.808	—/—
	4	.449/.681	.336/.574	.747/.870	.850/.927	.775/.891	.783/.881	.747/.890	—/—
GIST	1	.971/.889	—/—	.974/.889	.993/.902	.987/.889	.974/.889	.984/.887	.964/.894
	2	.988/.906	.973/.894	.984/.941	.995/.956	.994/.940	.991/.940	.989/.945	.996/.949
	3	.989/.936	.988/.909	.996/.967	.996/.977	.999/.971	.996/.969	.996/.973	—/—
	4	.993/.961	.989/.938	.998/.980	.999/.988	.999/.981	.998/.982	.998/.984	—/—
DBpedia-OpenAI	1	.756/.745	—/—	.740/.745	.736/.776	.720/.752	.740/.746	.702/.751	.728/.752
	2	.750/.784	.762/.753	.851/.868	.871/.897	.851/.864	.839/.869	.851/.876	.861/.883
	3	.806/.861	.752/.789	.909/.928	.945/.949	.908/.934	.918/.930	.911/.936	—/—
	4	.882/.921	.812/.864	.937/.960	.965/.973	.948/.959	.947/.962	.955/.964	—/—

Table 20 Trellis memory at one bit (SIFT-128, 1,000 queries, exact-cosine gold, uncentered). Every row from $M = 12$ to 16 has the same 22-byte record.

M	state B	record B	R@10	vs $M = 12$
8	1	21	.2825	−2.75
12	2	22	.3100	+0.00
14	2	22	.3214	+1.14
16	2	22	.3225	+1.25

question under the flat comparison’s own protocol on all three corpora, and the gain is larger there than the sweep above suggests. Against $M = 12$, Recall@10 rises 1.5 and 1.0 points at one and two bits on SIFT, 1.5 at one bit on GIST — enough to turn that cell from one this codec did not hold into one it holds with a paired interval excluding zero — and between -0.1 and $+0.5$ across DBpedia-OpenAI. Nine of the twelve cells improve, one is unchanged, and two go the other way: two-bit GIST by 0.7 points and two-bit DBpedia-OpenAI by 0.1. Those two and the graph study of Section 8 are reported at the same memory as the rest of the flat and systems tables rather than tuned around.

B.3 What the memory lever costs to encode

Trellis memory is free in emitted bits and free in record size within a byte tier, so the entire price is paid by the encoder. Table 21 measures it on the corpus and grid the recall sweep uses, which makes the two tables readable together: Figure 2 gives what a cell buys and this one gives what it costs. Encoding is an index-build operation performed once per vector, not per query.

Exact encoding is $O(D2^M2^{b_s})$, so each two-bit step of M should cost four times as much, and through $M = 14$ it does: measured consecutive steps span 3.2 – $5.0\times$ at every rate once $M \geq 8$, falling below the model only at small M where per-vector fixed costs dilute it.

The step into the reported $M = 16$ keeps that shape at the low rates and leaves it at the top one: $4.20\times$ at two bits and $4.30\times$ at three, against $9.62\times$ at four, where the 2^{M+b_s} -float table reaches 4 MiB and the encoder’s gather leaves cache — the boundary Section 10 reports for the graph scan, met on the ingest side. Ingesting a million vectors takes about sixteen minutes at two bits against a minute at $M = 12$, and about three hours at four.

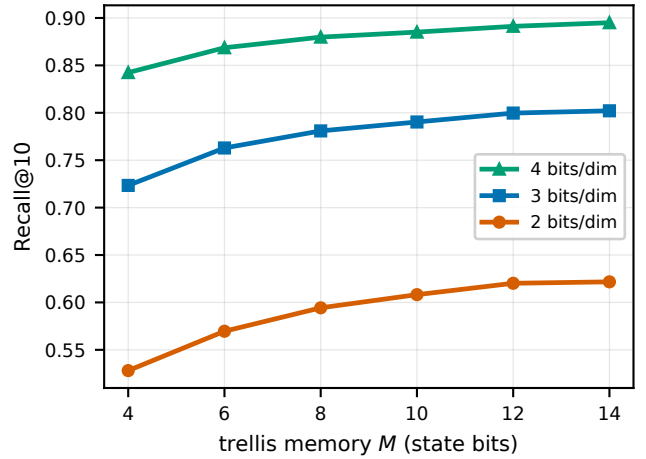


Fig. 2 Recall@10 against trellis memory M at each emitted rate (SIFT-100k, 1,000 queries, exact Viterbi). The emission payload is constant along each curve; the full record gains one byte when M crosses eight bits, while exact encoding cost grows as 2^M . The sweep runs $M = 4$ through 14; recall improves across the whole range and flattens at $M = 12$ at this rate, which is what makes the further step to the reported $M = 16$ cost encode time and little else. Drawn from the retained ablation bundle, which carries the per-cell values.

The steps below that one were not always at the model either, and there the gap was implementation rather than algorithm: a per-vector zeroed allocation of the backpointer array, a full 32-bit window stored where the emission index suffices, and a CPU feature check consulted twice per column. Removing all three changes no emitted bit and takes $M = 14$ from 2,120 to 4,260 vectors per second. At $4.11\times$ per two-bit step rather than the $8.5\times$ the unoptimized encoder measures, $M = 16$ is affordable to evaluate, which is why this paper reports it.

B.4 Is the scalar trellis the right member of its family?

Every block-based comparator in Table 3 quantizes coordinates jointly — PQ over subspaces, E8 and BlockQuant over blocks — and that is where a block code’s shape gain comes from. UltraVec is reported throughout at $V = 1$, one coordinate per step, so the obvious question is whether it is being compared

Table 21 Encode throughput against trellis memory at a fixed emitted rate (normalized SIFT-128, 100,000 vectors, exact Viterbi, 20 threads, vectors per second). Best of at least three timed encodes per cell; the median cell varied by $1.02\times$ across its own samples and the worst by $1.12\times$, so differences below about a fifth are not resolved. As elsewhere, the host is shared-tenancy and absolute rates are machine-specific — the memory-step ratios in the text are the portable quantity.

M	2 bits	3 bits	4 bits
4	2,020,000	1,460,000	986,000
6	917,000	528,000	314,000
8	287,000	157,000	81,800
10	72,400	36,900	19,200
12	17,500	9,120	4,720
14	4,260	2,080	944
16	1,020	483	98

in its weakest form. The construction supports $V > 1$: a step then emits bV bits and reconstructs V coordinates over D/V steps, so the payload is Db bits for every V and only the cell shape changes.

It buys nothing that it does not immediately give back. The sweep runs $V = 1, 2, 4$ at two bits and $V = 1, 2$ at three and four, at a payload identical to $V = 1$ in every case; it stops where the 2^{M+bV} reconstruction table does, not where the emission bound $bV \leq 16$ does, since the next cell up at either rate needs a table of gigabytes. The two most precise comparisons, at three and four bits where sampling noise is smallest, move by -0.10 points; the two-bit pair straddles zero at ± 0.70 , and Recall@1 moves by $+1.5$, $+1.1$, -0.6 and $+2.2$ with no consistent sign. Encode throughput falls by 20 to 142 times, since the table grows by 2^b with every step of V . The lever is already pulled: a block code has one source of joint structure, its block dimension, while a trellis carries 2^M states of memory between emissions — which Section 6 shows is what moves direction fidelity. The scalar trellis is the operating point of its family.

B.5 Additional fixed-record levers

We evaluate two further calibration-free, fixed-record knobs available in the implementation.

The shared rotation composes R orthonormal rounds with fresh signs. At $M = 12$ and two bits on SIFT, Recall@10 is 0.6151, 0.6198, 0.6202 and 0.6168 for $R = 1, 2, 3, 4$, with a peak at the three-round operating point. Because rotation is shared preprocessing, this result applies to the full codec field, and the field is measured rather than assumed: going from one round to three, E8 gains 2.20 Recall@10 points and PVQ 1.00 against the trellis’s 0.60, so the trellis’s margin over the best comparator narrows from $+6.60$ to $+5.50$ points. We adopt three rounds knowing they narrow our own margin, because they are the better transform for the field.

The anisotropic branch metric weights the Viterbi cost of coordinate i by $1 + \lambda x_i^2$, emphasizing coordinates where the encoded vector carries most energy. Recall@10 decreases monotonically, 0.6202, 0.6059, 0.5935 and 0.5759 for $\lambda = 0, 0.5, 1, 2$. The reason is structural. The Hadamard rotation leaves coordinates close to exchangeable, so per-coordinate magnitude carries little information about retrieval-relevant error, and the weighting shifts the metric away from the unweighted operating point. This differs from the anisotropic decomposition of

ScaNN [16], which separates error parallel to the datapoint from error orthogonal to it and requires a vector-level decomposition beyond this per-coordinate branch metric.

The residual control variate is the one lever here that changes the *estimator* rather than the code. Quantize as usual, store the residual’s norm and the sign bits of an independent sketch of it, and correct the inner product by $\sqrt{\pi/2} m^{-1} \|r\| \langle \text{sketch}(q), \text{signs} \rangle$, which is unbiased for the residual’s contribution. It is the technique the TurboQuant inner-product arm of Table 3 uses, so the question is whether it also pays on a trellis base. The correction must be taken against the reconstruction the scorer actually uses: scoring returns $\langle q_r, \bar{x} \rangle / \rho$, so the residual belongs against $\bar{x} / \rho$ and not against the naive $\sigma \bar{x}$; composed against the latter it replaces a near-unbiased estimator with a biased one. Composed correctly, what it buys is measured below in recall rather than in estimator spread.

What it buys is real and still not worth its bytes. At a fixed base rate the correction gains 3.60, 2.70, 1.20 and 0.70 Recall@10 points at one through four bits, for twenty bytes per vector — largest where the base is coarsest, which is what the variance of a sketched correction predicts. Traded bit-for-bit against a finer base it loses 27.60, 15.30 and 7.70 points. It is dominated even on bytes: the corrected one-bit arm spends 46 bytes for 0.3440 where the plain two-bit codec spends 42 for 0.6200, and the same holds at every rate measured. On this codec the bits belong in the base. That is the informative outcome rather than a null one, because the identical construction gains 4.8 points on TurboQuant’s scalar base at two bits: a base that already codes the direction well leaves a residual too small to be worth correcting, and the trellis is that base.

The two-stage shortlist has its own estimator choice, and it is not a small one. Its first stage compares a query against one-bit sign codes, which can be done with the query in full precision, scoring $\langle q_r, \text{sign}(u_r) \rangle$, or with the query binarized too and ranked by Hamming distance. Keeping the query is worth 6.80, 3.50 and 1.55 Recall@10 points at $C = 128, 256$ and 512, the gap closing as the shortlist stops binding — and it costs most of the scan speed, running at 0.19, 0.27 and 0.38 times the binarized rate, because it does $O(D)$ float work per entry where the other does a popcount. Neither dominates: the two trace one frontier. We report the full-precision end because the shortlist exists to raise recall; the binarized one is the choice when throughput binds. Under centering a second term appears: a stored vector is $c + \|u - c\|r$ and only r is sign-coded, so the per-vector factor $\|u - c\|$ has to be reapplied in the first stage. Doing so adds 0.60, 0.35 and 0.05 points with the full-precision scorer and 0.55, 0.20 and 0.15 with the binarized one. Both are enabled in the reported arm.

Two further knobs are reserved for fitted variants: a learned PCA or ITQ rotation and centering against a corpus mean. Both can raise recall by estimating parameters from the corpus, while the plain codec is defined by calibration-free operation. The PCA-head residual hybrid of Section 3.4 is where fitted state is evaluated.

C Artifact and Evidence Provenance

Each enabled claim maps to bundles named under the artifact’s `results/generated` directory. A bundle records the literal command, retained output, dataset and baseline hash maps, campaign-start and completion revisions, computation-tree hash, toolchain, CPU, threads, seed and status, and the runner requires a clean initial tree. Each tier declares its own operating point — state memory, thread counts, any

codec knob it varies — and the runner refuses to inherit an undeclared one, so a bundle records a protocol rather than a shell.

Validation comes in two strengths, because the released artifact is a fresh repository rather than a copy of the authoring history. With that history present, strict mode recomputes each bundle’s computation-tree hash *at* its recorded revision, binding the retained numbers to the code that produced them; without it the portable check verifies pins, seeds and the clean-tree conditions. Dataset hashes are re-verified for whichever corpora a reader has staged. Strict validation is run from the authoring repository before every release.

The official RaBitQ source uses `std::random_device` in its FHT sign rotation, so the preparation script applies a one-line seed-42 patch whose hash the baseline manifest records, and requires byte-identical E-RaBitQ output across two runs before downstream intervals are admitted.

References

- Aden-Ali, I., Ferhatosmanoglu, H., Greaves-Tunnell, A., Mishra, N., Wagner, T.: Quantization for vector search under streaming updates. arXiv preprint arXiv:2512.18335 (2025)
- Aguerreberre, C., Bhati, I.S., Hildebrand, M., Tepper, M., Willke, T.: Similarity search in the blink of an eye with compressed indices. Proc. VLDB Endowment **16**(11), 3433–3446 (2023). ArXiv:2304.04759
- Aguerreberre, C., Hildebrand, M., Bhati, I.S., Willke, T., Tepper, M.: Locally-adaptive quantization for streaming vector search. arXiv preprint arXiv:2402.02044 (2024)
- André, F., Kermarrec, A.M., Le Scouarnec, N.: Cache locality is not enough: High-performance nearest neighbor search with product quantization fast scan. Proceedings of the VLDB Endowment **9**(4), 288–299 (2015). <https://doi.org/10.14778/2856318.2856324>
- André, F., Kermarrec, A.M., Le Scouarnec, N.: Accelerated nearest neighbor search with quick adc. In: Proceedings of the ACM on International Conference on Multimedia Retrieval (ICMR), pp. 159–166 (2017). <https://doi.org/10.1145/3078971.3078992>
- Ann, H., Lee, J., Oh, M.H.: Block-sphere vector quantization. arXiv preprint arXiv:2605.19972 (2026)
- Aumüller, M., Bernhardtsson, E., Faithfull, A.: ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms. Information Systems **87**, 101,374 (2020). <https://doi.org/10.1016/j.is.2019.02.006>. ArXiv:1807.05614
- Choi, J., Chance, Z., Love, D.J., Madhow, U.: Noncoherent trellis coded quantization: A practical limited feedback technique for massive MIMO systems. IEEE Trans. Commun. (2013). ArXiv:1305.4976
- Conway, J.H., Sloane, N.J.A.: Sphere Packings, Lattices and Groups, 3rd edn. Springer (1999)
- DBpedia entities OpenAI 1m embeddings. HuggingFace dataset KShivendu/dbpedia-entities-openai-1M (2023). OpenAI text-embedding-ada-002, 1536-d
- Fischer, T.R.: A pyramid vector quantizer. IEEE Transactions on Information Theory **32**(4), 568–583 (1986)
- Gao, J., Gou, Y., Xu, Y., Shi, J., Yang, Y., Li, S., Wong, R.C.W., Long, C.: Revisiting RaBitQ and TurboQuant: A symmetric comparison of methods, theory, and experiments. arXiv preprint arXiv:2604.19528 (2026)
- Gao, J., Gou, Y., Xu, Y., Yang, Y., Long, C., Wong, R.C.W.: Practical and asymptotically optimal quantization of high-dimensional vectors in Euclidean space for approximate nearest neighbor search. Proc. ACM Manag. Data **3**(3), 1–26 (2025). <https://doi.org/10.1145/3725413>. ArXiv:2409.09913
- Gao, J., Long, C.: RaBitQ: Quantizing high-dimensional vectors with a theoretical error bound for approximate nearest neighbor search. In: Proc. ACM SIGMOD (2024). ArXiv:2405.12497
- Ge, T., He, K., Ke, Q., Sun, J.: Optimized product quantization for approximate nearest neighbor search. In: Proc. IEEE CVPR, pp. 2946–2953 (2013). <https://doi.org/10.1109/CVPR.2013.379>
- Guo, R., Sun, P., Lindgren, E., Geng, Q., Simcha, D., Chern, F., Kumar, S.: Accelerating large-scale inference with anisotropic vector quantization. In: Proc. ICML (2020). ArXiv:1908.10396
- Huijben, I.A.M., Douze, M., Muckley, M., van Sloun, R.J.G., Verbeek, J.: Residual quantization with implicit neural codebooks. In: Proc. ICML (2024). ArXiv:2401.14732
- Iyengar, V.: BCJR-QAT: A differentiable relaxation of trellis-coded weight quantization. arXiv preprint arXiv:2605.10655 (2026)
- Jääsaari, E., Hyvönen, V., Ceccarello, M., Roos, T., Aumüller, M.: VIBE: Vector index benchmark for embeddings. arXiv preprint arXiv:2505.17810 (2025)
- Jégou, H., Douze, M., Schmid, C.: Product quantization for nearest neighbor search. IEEE Trans. Pattern Anal. Mach. Intell. **33**(1), 117–128 (2011)
- Jégou, H., Tavenard, R., Douze, M., Amsaleg, L.: Searching in one billion vectors: Re-rank with source coding. In: Proc. IEEE ICASSP, pp. 861–864 (2011). <https://doi.org/10.1109/ICASSP.2011.5946540>. ArXiv:1102.3828
- Khan, M.A.U., Smith, M.J.T., McLaughlin, S.W.: Trellis-coded residual vector quantization: Its geometrical advantages and application to image coding. In: Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP), vol. 5, pp. 2617–2620 (2000). <https://doi.org/10.1109/ICASSP.2000.861005>
- Li, H., Deng, S., Yan, X., Zhi, X., Cheng, J.: SAQ: Pushing the limits of vector quantization through code adjustment and dimension segmentation. Proc. ACM Manag. Data **3**(6), 1–25 (2025). <https://doi.org/10.1145/3769824>. ArXiv:2509.12086
- Malkov, Y.A., Yashunin, D.A.: Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. IEEE Trans. Pattern Anal. Mach. Intell. **42**(4) (2020). ArXiv:1603.09320
- Mao, M.Z., Gray, R.M.: Stationary and trellis encoding for IID sources and simulation. In: Proc. Data Compression Conference, pp. 3–12 (2010). <https://doi.org/10.1109/DCC.2010.8>
- Marcellin, M.W., Fischer, T.R.: Trellis coded quantization of memoryless and Gauss-Markov sources. IEEE Trans. Commun. **38**(1), 82–93 (1990). <https://doi.org/10.1109/26.46532>
- Ordentlich, O., Polyanskiy, Y.: Optimal quantization for matrix multiplication. IEEE Trans. Inf. Theory **72**(3), 1943–1972 (2026). ArXiv:2410.13780
- Pennington, J., Socher, R., Manning, C.D.: GloVe: Global vectors for word representation. In: Proc. EMNLP (2014)
- Savkin, S., Porat, E., Ordentlich, O., Polyanskiy, Y.: NestQuant: Nested lattice quantization for matrix products and LLMs. In: Proc. ICML (2025). ArXiv:2502.09720
- Sharma, T.: IVF-TQ: Calibration-free streaming vector search via a codebook-free residual layer. arXiv preprint arXiv:2605.17415 (2026)
- Shi, J., Gao, J., Xia, J., Féhé, T.B., Long, C.: GPU-native approximate nearest neighbor search with IVF-RaBitQ: Fast index build and search. arXiv preprint arXiv:2602.23999 (2026)
- Simhadri, H.V., Williams, G., Aumüller, M., Douze, M., Babenko, A., Baranchuk, D., Chen, Q., Hosseini, L., Krishnaswamy, R., Srinivasa, G., Subramanya, S.J., Wang, J.: Results of the NeurIPS’21 challenge on billion-scale approximate nearest neighbor search. In: Proc. NeurIPS Competitions and Demonstrations Track (PMLR 176), pp. 177–189 (2022). ArXiv:2205.03763
- Subramanya, S.J., Devvrit, Simhadri, H.V., Krishnaswamy, R., Kadekodi, R.: DiskANN: Fast accurate billion-point nearest neighbor search on a single node. In: Proc. NeurIPS (2019)
- Tepper, M., Willke, T.: Individualized non-uniform quantization for vector search. arXiv preprint arXiv:2509.18471 (2025)
- Tseng, A., Chee, J., Sun, Q., Kuleshov, V., De Sa, C.: QuIP#: Even better LLM quantization with Hadamard incoherence and lattice codebooks. In: Proc. ICML (2024). ArXiv:2402.04396
- Tseng, A., Sun, Q., Hou, D., De Sa, C.: QTIP: Quantization with trellises and incoherence processing. In: Proc. NeurIPS (2024). ArXiv:2406.11235
- Ungerboeck, G.: Channel coding with multilevel/phase signals. IEEE Trans. Inf. Theory **28**(1), 55–67 (1982). <https://doi.org/10.1109/TIT.1982.1056454>
- Vallaeys, T., Muckley, M., Verbeek, J., Douze, M.: QInco2: Vector compression and search with improved implicit neural codebooks. In: Proc. ICLR (2025). ArXiv:2501.03078
- Vargaftik, S., Ben-Basat, R., Portnoy, A., Mendelson, G., Ben-Itzhak, Y., Mitzenmacher, M.: EDEN: Communication-efficient and robust distributed mean estimation for federated learning. In: Proc. ICML (2022). ArXiv:2108.08842
- Wei, B., Guan, T., Yu, J.: Projected residual vector quantization for ANN search. IEEE MultiMedia **21**(3), 41–51 (2014)
- Xiao, W., Wang, Z., Li, C.: Quiver: Rethinking ANN graph topology via training-free binary quantization. arXiv preprint arXiv:2605.02171 (2026)
- Yang, M., Jing, L., Li, W., Wang, W.: Quantization meets projection: A happy marriage for approximate k-nearest neighbor search. Proc. VLDB Endow. **19**(6), 1240–1249 (2026). <https://doi.org/10.14778/3797919.3797931>. ArXiv:2411.06158
- Yang, Z., Li, X., Cheng, K., Liu, K., Yang, J., Wang, H., Deng, K., Dang, Q., Sha, Y., Ma, Y., Yu, D.: Proteus: Lookup-free trellis-coded quantization by lattice-breaking compute codes for 2-bit LLMs. In: Proc. ICML (2026)
- Zandieh, A., Daliri, M., Hadian, M., Mirrokni, V.: TurboQuant: Online vector quantization with near-optimal distortion rate. In: Proc. ICLR (2026). ArXiv:2504.19874
- Zhang, J., Liu, Q., Lian, D., Liu, Z., Wu, L., Chen, E.: Anisotropic additive quantization for fast inner product search. In: Proc. AAAI (36/4), pp. 4354–4362 (2022). <https://doi.org/10.1609/aaai.v36i4.20356>